



Tribhuvan University
Faculty of Humanities and Social Sciences

A PROJECT REPORT ON
“BeemaSathi: A Personalized Insurance System”

Submitted to:
Department of Computer Application
JANA BHAWANA CAMPUS

In partial fulfillment of the requirement for the degree of
Bachelor of Computer Application

Sushil Timalsina
T.U. Reg. No.: 6-2-253-27-2022

Under the Supervision of
Mr. Santosh Pyakhurel

Abstract

This project report presents the design and implementation of BeemaSathi: A Personalized Insurance System, a modern full-stack fintech platform developed to simplify the discovery, comparison and selection of insurance policies. The system addresses the common challenges of information overload, non-transparent pricing, and complex underwriting processes that users typically face in the insurance industry.

BeemaSathi introduces a data-driven recommendation engine and a dynamic actuarial premium calculator to provide users with personalized policy suggestions based on their individual risk profile, including age, health indicators, lifestyle habits, and regional factors. Instead of relying on static price listings, the platform calculates real-time premiums using a risk-based multiplicative model, reflecting practical insurance underwriting principles.

The system is developed using a decoupled architecture, with Laravel (PHP) powering the backend RESTful API and React.js delivering a responsive Single Page Application (SPA) on the frontend. Security and integrity are maintained through role-based access control, session monitoring mechanisms, and synchronized authentication across multiple browser tabs.

The report includes system architecture, database schema design, algorithmic models, and implementation details covering policy management, premium calculation, recommendation logic, administrative controls, and secure session handling. By integrating algorithmic personalization, pricing transparency, and a user-centric interface, BeemaSathi transforms insurance selection from a confusing manual process into an intelligent and streamlined digital experience.

This project demonstrates the practical application of software engineering, database management systems, risk modeling concepts, and modern web technologies in solving a real-world financial services challenge, while maintaining scalability for future enhancements such as AI driven underwriting and advanced analytics.

Acknowledgement

It is my pleasure to express my sincere gratitude to all those who supported and guided me throughout the development of this project.

First and foremost, I would like to express my deepest gratitude to my project supervisor, **Mr. Santosh Pyakhurel**, for his continuous guidance, valuable suggestions, and constructive feedback throughout the project. His support greatly helped in improving both the technical and documentation aspects of this work.

I would also like to express my sincere thanks to the Campus Chief of Jana Bhawana Campus, **Mr. Shivraj Sanjel**, and the Department of Computer Application (BCA) for providing the necessary academic environment, resources, and institutional support for the successful completion of this project.

Finally, I would like to thank all faculty members, as well as my friends and family, for their continuous encouragement, support, and motivation throughout my BCA program and during the completion of this project.

Table of Contents

Abstract	I
Acknowledgement.....	II
LIST OF ABBREVIATIONS	V
LIST OF FIGURES	VI
LIST OF TABLES	VII
Chapter 1: Introduction.....	1
1.1 Introduction	1
1.2 Problem statement	2
1.3 Objectives	3
1.4 Scope and Limitation.....	3
1.5 Development Methodology	4
1.6 Report Organization	5
Chapter 2: Background Study and Literature Review.....	6
2.1 Background Study	6
2.2 Literature Review	6
Chapter 3: System Analysis and Design.....	9
3.1 System Analysis.....	9
3.1.1 Requirement Analysis.....	9
3.1.2 Feasibility Analysis	12
3.1.3 Object modelling: Object & Class Diagram.....	14
3.1.4 Dynamic Modelling: State & Sequence diagram	17
3.2 System Design.....	23
3.2.1 Refinement of Classes and Object.....	24
3.2.2 Component diagram	27
3.2.3 Deployment diagram	29
3.3 Algorithm.....	30
3.3.1 Adaptive Hybrid Policy Matcher (AHPM).....	30
3.3.2 Dynamic Actuarial Pricing Engine (DAPE) Algorithm.....	33

Chapter 4: Implementation and Testing.....	35
4.1 Implementation.....	35
4.1.1 Tools Used (CASE Tools, Programming Language, Database Platforms).....	35
4.1.2 Implementation details of modules.....	37
4.2 Testing	41
4.2.1 Test case for Unit Testing	41
4.2.2 Test case for System Testing.....	47
Chapter 5: Conclusion and Future Recommendations	50
5.1 Conclusion.....	50
5.3 Future Recommendations	51
REFERENCES	52
APPENDICES.....	53

LIST OF ABBREVIATIONS

AHPM:	Adaptive Hybrid Policy Matcher
API:	Application Programming Interface
BMI:	Body Mass Index
CRUD:	Create, Read, Update, and Delete
CSS:	Cascading Style Sheets
DAPE:	Dynamic Actuarial Pricing Engine
DBMS:	Database Management System
HTML:	HyperText Markup Language
HTTP / HTTPS:	Hypertext Transfer Protocol (Secure)
JS:	JavaScript
JWT:	JSON Web Token
KYC:	Know Your Customer
MVC:	Model-View-Controller
MySQL:	My Structured Query Language
ORM:	Relational Mapping
OTP:	One-Time Password
PHP:	Hypertext Preprocessor
REST:	Representational State Transfer
SaaS:	Software as a Service
SMTP:	Simple Mail Transfer Protocol
SPA:	Single Page Application
SQL:	Structured Query Language
UI/UX:	User Interface / User Experience

LIST OF FIGURES

Figure 1. 1 Agile Methodology	4
Figure 3. 2: Use Case Diagram for BeemaSathi	11
Figure 3. 3: Gantt Chart of BeemaSathi.....	13
Figure 3. 4: Object Diagram of BeemaSathi.....	14
Figure 3. 5: Class Diagram of BeemaSathi.....	15
Figure 3. 6: State Diagram of Policy Purchase Lifecycle in BeemaSathi	17
Figure 3. 7: Client Sequence Diagram of BeemaSathi	19
Figure 3. 8: Admin Sequence Diagram of BeemaSathi	20
Figure 3. 9: Process modelling: Activity Diagram.....	22
Figure 3. 10: Refinement of Classes for BeemaSathi	24
Figure 3. 11: Refinement of Object Diagram for BeemaSathi	26
Figure 3. 12: Component Diagram of BeemaSathi for User	27
Figure 3. 13: Component Diagram of BeemaSathi for Admin	28
Figure 3. 14: Deployment Diagram for BeemaSathi	29
Figure 3. 15: Adaptive Hybrid Policy Matcher (AHPM) Diagram	32
Figure 3. 16: Dynamic Actuarial Pricing Engine (DAPE) Algorithm Diagram	34

LIST OF TABLES

Table 4. 1: Test Case for User Registration of BeemaSathi	41
Table 4. 2: Test Case for User Login of BeemaSathi	43
Table 4. 3: Test Case for Admin Module of BeemaSathi	44
Table 4. 4: Test Case for System Testing of Beema Sathi	47

REPORT II

Chapter 1: Introduction

1.1 Introduction

In recent years, the insurance sector has undergone a significant transformation driven by the rapid adoption of digital technologies. Traditionally, purchasing and managing insurance policies in Nepal has been a complex, paper-intensive process that requires multiple visits to insurance offices, lengthy documentation, and heavy reliance on intermediaries such as agents and brokers. This conventional approach often results in a lack of transparency, difficulty in comparing policies, and limited accessibility for the general public particularly for those in remote areas or those unfamiliar with the intricacies of insurance products.

BeemaSathi is a full-stack insurance marketplace and management platform designed to address these challenges by providing a centralized digital environment where users can browse insurance policies, receive personalized premium estimates, complete KYC verification, submit purchase requests, and make payments all through a single, unified web-based interface. The name "BeemaSathi" directly reflects its purpose: to be a trusted companion ("Sathi") in the insurance ("Beema") journey of every user.

The platform supports three distinct user experiences. First, it allows public visitors to browse available policies and obtain premium quotes without requiring registration. Second, it provides a comprehensive client self-service portal where registered users can manage their policies, track renewals, handle support tickets, and monitor payments. Third, it offers a powerful administrative console through which administrators can manage policies, agents, insurance companies, clients, KYC approvals, reports, and audit logs.

BeemaSathi is built using a modern technology stack. The backend is developed with Laravel, a robust PHP framework, and exposes REST APIs secured with Laravel Sanctum token-based authentication. The frontend is a React Single Page Application built with Vite and styled using Tailwind CSS, consuming the backend APIs through Axios. Payments are handled through two popular Nepali payment gateways eSewa and Khalti making the platform directly relevant to the Nepali digital financial ecosystem.

The system incorporates several intelligent features including a rule-based premium calculator that estimates insurance costs based on user profile factors such as age, smoking habits, health score, and coverage type. A lead distribution mechanism automatically assigns the most available insurance agent to new purchase requests, ensuring efficient workload management. The platform also includes automated renewal processing through a scheduled command that tracks due dates and sends timely reminders to users.

Overall, BeemaSathi aims to modernize the insurance experience in Nepal by making it more accessible, transparent, and efficient for clients, while simultaneously providing administrators and agents with powerful tools to manage operations effectively.

1.2 Problem statement

The insurance industry in Nepal faces several deep-rooted challenges that hinder its widespread adoption and effective management. Customers often struggle to find, compare, and understand insurance policies due to the fragmented and manual nature of the existing system. The absence of a centralized digital platform means that potential buyers must rely entirely on agents or visit multiple insurance companies to gather basic product information, creating significant inefficiency and information asymmetry.

Furthermore, the processes of KYC verification, policy purchase, and premium payment are typically handled through physical paperwork, making them time-consuming and prone to errors. Renewal tracking is often neglected, leading to policy lapses that leave clients unprotected. Agents and administrators lack a unified system to manage client requests, track payments, and monitor renewals effectively, resulting in operational bottlenecks and poor customer service.

There is therefore a pressing need for a secure, integrated, and user-friendly digital platform that can streamline the end-to-end insurance lifecycle from policy discovery and premium calculation to KYC submission, payment processing, and renewal management for both clients and administrators.

1.3 Objectives

The main objectives of this project are as follows:

- To develop a public-facing insurance marketplace where users can browse and compare insurance policies and obtain personalized premium estimates.
- To implement a secure client portal that allows users to register, complete KYC, submit policy purchase requests, and make payments via eSewa and Khalti.
- To build a comprehensive admin console for managing policies, agents, companies, clients, KYC approvals, renewals, payments, and audit logs.
- To automate renewal tracking and notifications using a scheduled backend process.
- To implement a rule-based premium calculation engine that accounts for user profile factors such as age, smoking status, health score, and coverage type.
- To provide a support ticket system enabling seamless communication between clients and administrators.

1.4 Scope and Limitation

a. Scope

- Registered users can browse insurance policies, request premium quotes, submit KYC documents, purchase policies, and make payments through the platform.
- The system supports two Nepali payment gateways eSewa and Khalti enabling real digital transactions.
- Administrators can manage the full lifecycle of insurance products including policies, agents, companies, clients, KYC approvals, payments, renewals, and audit logs through a dedicated admin console.
- The platform includes automated renewal tracking with email notifications and a support ticketing system for client-administrator communication.
- The system provides personalized policy recommendations based on user profile data using a custom rule-based algorithm.

b. Limitations

- The platform is designed primarily for the Nepali market and supports only eSewa and Khalti payment gateways; international payment methods are not included.

- All recommendation, comparison, and premium calculation features are based on custom rule-based logic and do not employ machine learning or external algorithmic APIs.
- The system requires a stable internet connection for API communication, payment verification, and email notifications; offline functionality is not supported.
- Biometric verification and integration with government identity systems for automated KYC validation are not implemented in the current version.

1.5 Development Methodology

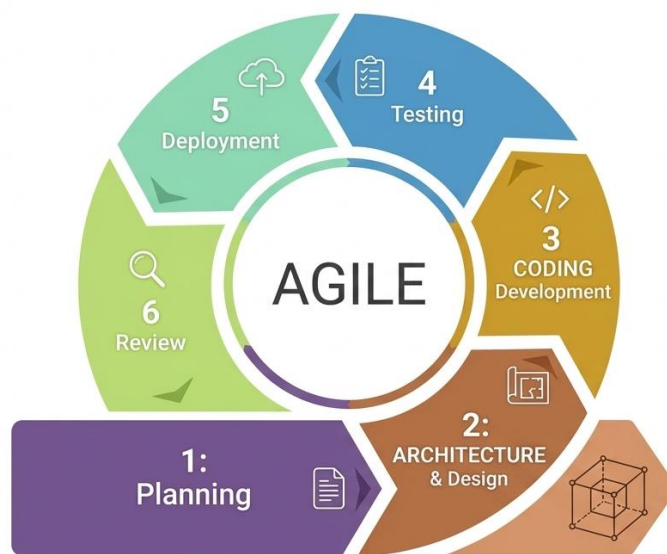


Figure 1. 1 Agile Methodology

BeemaSathi was developed using the Agile methodology, which emphasizes iterative development, continuous testing, and flexibility in adapting to changing requirements. Given the complexity of the platform involving multiple user roles, payment gateway integrations, and a wide range of features Agile was the most appropriate approach as it allowed the development team to build, test, and refine individual modules incrementally rather than waiting until the entire system was complete.

Each iteration focused on a specific set of features, such as authentication, KYC management, payment processing, or the admin console. This allowed potential issues to be identified and resolved early in the development process, reducing the risk of critical failures in later stages. Regular reviews of completed modules ensured that the system

aligned with both functional requirements and user expectations throughout the development cycle.

The Agile approach also made it easier to manage the integration of external services such as eSewa and Khalti, where API behavior needed to be tested iteratively, and adjustments to callback handling and verification logic were made progressively based on test outcomes.

1.6 Report Organization

Introduction This chapter provides an overview of the BeemaSathi platform, including its purpose, problem statement, objectives, scope, limitations, and development methodology.

Background Study and Literature Review This chapter reviews existing insurance marketplace platforms and relevant literature on digital insurance systems, highlighting the gaps that BeemaSathi aims to address.

System Analysis and Design This chapter covers the functional and non-functional requirements of the system, feasibility analysis, Entity Relationship Diagram (ERD), Data Flow Diagram (DFD), system architecture, database schema design, and interface design.

Implementation and Testing This chapter describes the tools and technologies used in the development of BeemaSathi, the implementation details of key modules, and the results of unit and system testing performed to validate the platform's functionality.

Conclusion and Future Recommendations This chapter summarizes the outcomes and lessons learned from the project, and outlines potential future enhancements such as machine learning-based recommendations, biometric KYC, and mobile application development.

Chapter 2: Background Study and Literature Review

2.1 Background Study

Before developing BeemaSathi, a study of the existing insurance landscape in Nepal and current digital insurance platforms was conducted to identify key shortcomings and understand how technology could address them.

In Nepal, despite a growing number of registered insurance companies regulated by the Insurance Regulatory Authority (Beema Pradhikarana), the penetration rate of insurance remains low. Most citizens have limited knowledge of available policies and rely entirely on physical agents as their sole point of contact. This dependency creates an information gap that discourages many potential customers from entering the insurance market.

The processes of KYC submission, policy purchase, premium payment, and renewal tracking are predominantly manual and paper-based. Clients must visit offices, fill physical forms, and wait for processing. Renewal reminders are communicated informally through agents, causing many policies to lapse simply due to lack of timely notification.

On the administrative side, insurance companies and agents typically manage client data, payments, and renewals through disconnected tools such as spreadsheets and standalone databases, making it difficult to generate reports, track agent performance, and audit actions effectively.

These challenges formed the foundation upon which BeemaSathi was designed a platform that digitizes the client-facing aspects of insurance while also providing administrators and agents with a centralized management console.

2.2 Literature Review

A. Digital Transformation in the Insurance Industry

The global insurance sector is transitioning from traditional broker-driven models to digital-first platforms, commonly referred to as InsurTech. The integration of web-based technologies has significantly reduced operational costs while increasing accessibility and efficiency [1]. Unlike legacy systems that depend on manual documentation and

intermediaries, modern digital platforms enable real-time interaction between insurers and customers. This transformation is particularly impactful in emerging markets, where digitalization has been shown to enhance transparency, build trust, and improve insurance penetration rates [2].

B. Automated Underwriting and Premium Calculation

Underwriting remains a critical component of insurance risk assessment. Traditional underwriting approaches often rely on generalized risk categories, leading to inefficiencies such as flat pricing models. Research indicates that algorithmic and rule-based underwriting systems enable more precise risk evaluation through behavioral data, including lifestyle indicators such as body mass index (BMI), smoking habits, and occupation [3]. Automated premium calculation systems further streamline the process by generating real-time quotations, minimizing human error, and eliminating subjective bias in decision-making [4].

C. Personalized Recommendation Systems in Fintech

The increasing number of available insurance products often leads to decision fatigue among users. To address this, recommendation systems have been widely adopted in fintech applications. While collaborative filtering techniques analyze user behavior patterns, the insurance domain benefits more from hybrid approaches. Studies suggest that combining knowledge-based inputs (e.g., user demographics and health metrics) with collaborative filtering (peer behavior) results in higher recommendation accuracy and improved user satisfaction [5]. Such systems enable users to identify policies that align closely with their financial capacity and risk profile.

D. Security, Privacy, and KYC Compliance

Security and regulatory compliance are fundamental in digital financial platforms. The implementation of Know Your Customer (KYC) frameworks is essential to prevent fraud and ensure legal compliance. Multi-layered security mechanisms, including session-based authentication and document verification systems, have been recommended to enhance system resilience [6]. Additionally, techniques such as hashing identifiers to prevent direct exposure of database keys help mitigate risks like ID enumeration attacks. These practices contribute to strengthening backend security and protecting sensitive user information [7].

E. Digital Payment Segregation and Trust

The integration of third-party payment gateways plays a critical role in establishing trust in e-commerce and fintech platforms. By leveraging PCI-DSS compliant providers, platforms can securely process transactions without storing sensitive financial data locally [8]. This separation of application logic from payment processing reduces system vulnerability and ensures compliance with industry security standards. Such architectural decoupling is considered a best practice in modern fintech system design, enhancing both security and scalability [9].

F. PolicyBazaar as a Digital Insurance Aggregator

PolicyBazaar is a digital platform that aggregates insurance products from multiple providers, allowing users to compare and purchase policies through a single interface. It improves transparency, reduces information asymmetry, and simplifies policy selection through comparison-based features [10].

G. PolicyBazaar Market Impact

PolicyBazaar has enhanced insurance accessibility by digitizing the entire purchasing process, including policy comparison, issuance, and renewal. Its data-driven system improves user decision-making and increases efficiency in insurance distribution [10].

Chapter 3: System Analysis and Design

3.1 System Analysis

The System Analysis phase of BeemaSathi focuses on identifying and defining the system's requirements based on existing challenges in traditional insurance processes. Current systems are often manual, fragmented, and lack transparency, making policy comparison and management difficult for users.

The analysis outlines key functional requirements such as user authentication, policy comparison, premium calculation, KYC processing, and payment integration. Non-functional requirements include security, scalability, performance, and usability to ensure a reliable and efficient system.

Feasibility analysis confirms that the system is technically achievable using technologies like React, Laravel, and MySQL, while also being cost-effective and user-friendly. Overall, this phase establishes a clear foundation for designing a secure and scalable digital insurance platform.

3.1.1 Requirement Analysis

Functional Requirements

1. **User Registration and Authentication:** The system must allow users to register, log in, and log out securely. Client and admin authentication are handled separately using Laravel Sanctum token-based authentication stored in sessionStorage.
2. **Policy Browsing and Premium Calculation:** Public users must be able to browse available insurance policies and request premium estimates based on personal profile factors such as age, smoking status, health score, coverage type, and family members.
3. **KYC Submission and Verification:** Registered users must be able to submit KYC documents through the platform. Administrators must be able to review, approve, or reject submissions, with the ability to grant edit access when corrections are needed.

4. **Policy Purchase and Buy Requests:** Authenticated users must be able to submit policy purchase requests.
5. **Payment Processing:** The system must support payments via eSewa and Khalti, verify transactions through their respective callback and return URL mechanisms, and distinguish between initial purchase payments and subsequent renewal payments.
6. **Renewal Tracking:** The system must automatically track renewal due dates, mark overdue renewals, and send reminder notifications to clients through a scheduled backend command.
7. **Support Ticket System:** Users must be able to create support tickets and exchange messages with administrators. Read and unread states must be tracked for both parties.
8. **Dynamic Rating & Review System:** The Dynamic Rating & Review System acts as a trust-building feedback module that enhances transparency. It enforces verified purchases, allowing only actual buyers to submit reviews. The rating feature is integrated into the renewal workflow for better engagement. It also updates average ratings in real time, ensuring accurate and reliable feedback.
9. **Admin Console:** Administrators must be able to manage policies, agents, companies, clients, KYC records, payments, renewals, notifications, reports, and audit logs through a dedicated admin interface.

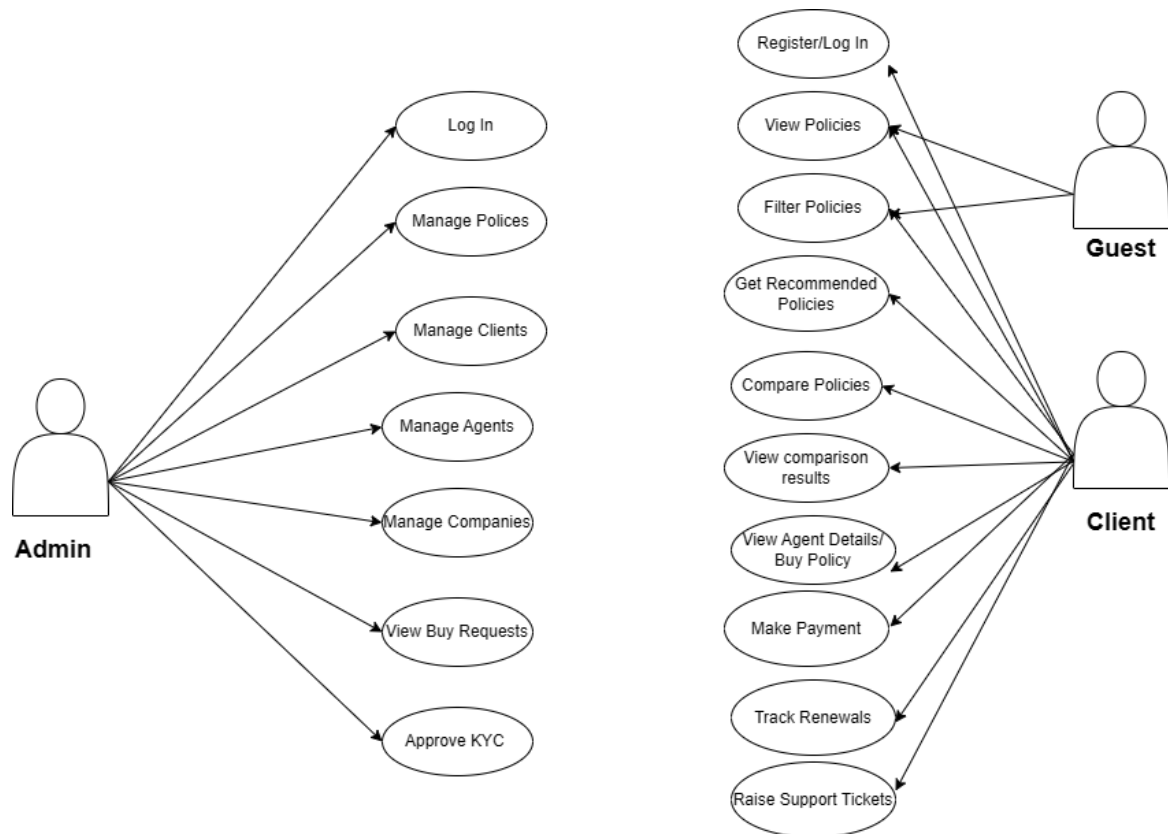


Figure 3. 2: Use Case Diagram for BeemaSathi

Here is the **Use Case Diagram** for BeemaSathi showing all three actors and their interactions with the system.

Actors:

- **Public User** can browse policies and get premium quotes without registering.
- **Client** registered user who can login, submit KYC, buy policies, make payments, track renewals, and raise support tickets.
- **Admin** manages policies, approves KYC, and manages agents and companies.

Non-Functional Requirements

1. **Security:** All API routes must be protected by Sanctum authentication tokens. Admin routes must additionally pass through AdminMiddleware. All admin actions must be recorded by AuditLogMiddleware.
2. **Reliability:** The system must handle payment callbacks reliably, ensure consistent renewal processing through scheduled commands, and maintain data integrity across all transactions.

3. **Usability:** The frontend must provide a clean, responsive user interface for both clients and administrators, built with React and Tailwind CSS for accessibility across devices.
4. **Scalability:** The system architecture must support the addition of new insurance companies, policy types, payment gateways, and administrative features without requiring significant restructuring.
5. **Performance:** API responses must be efficient, with Eloquent ORM queries optimized to handle growing data volumes for policies, payments, and client records.

3.1.2 Feasibility Analysis

i. Technical Feasibility

BeemaSathi is technically feasible as it is built using widely adopted and well-supported technologies. The backend uses Laravel, a mature PHP framework with built-in support for authentication, routing, middleware, scheduled commands, and Eloquent ORM. The frontend uses React 19 with Vite, Tailwind CSS, and Axios all modern, well-documented tools. Payment integration with eSewa and Khalti is achievable using their publicly available API documentation. The development environment uses XAMPP for local backend hosting, and all tools used are open source and freely available.

ii. Operational Feasibility

The system is operationally feasible for both clients and administrators. The client-facing interface is designed to be intuitive, guiding users through policy browsing, KYC submission, and payment in a step-by-step manner. The admin console provides a structured layout with clear navigation across all management modules. Both interfaces require minimal training and are accessible through any modern web browser without the need to install additional software.

iii. Economic Feasibility

The development of BeemaSathi involves minimal cost as all technologies used Laravel, React, MySQL, Tailwind CSS, and Vite are open source and freely available. Payment gateway integration with eSewa and Khalti can be initiated using their free developer sandbox environments. As this project is developed as part of the academic requirement for the Bachelor of Computer Application under Tribhuvan University, the primary costs are limited to hosting, printing, and minor operational expenses.

iv. Schedule Feasibility

The schedule feasibility for the Urban Glow Salon system is ensured through a well-structured Gantt chart where tasks are properly sequenced, and some phases overlap with efficiency. Key development stages such as implementation, testing, and documentation run in parallel, allowing for the timely completion of core booking and e-commerce modules. Adequate time is allocated to each phase to maintain a smooth technical workflow and ensure the integration of custom recommendation algorithms. Overall, the project is feasible within the given timeframe.

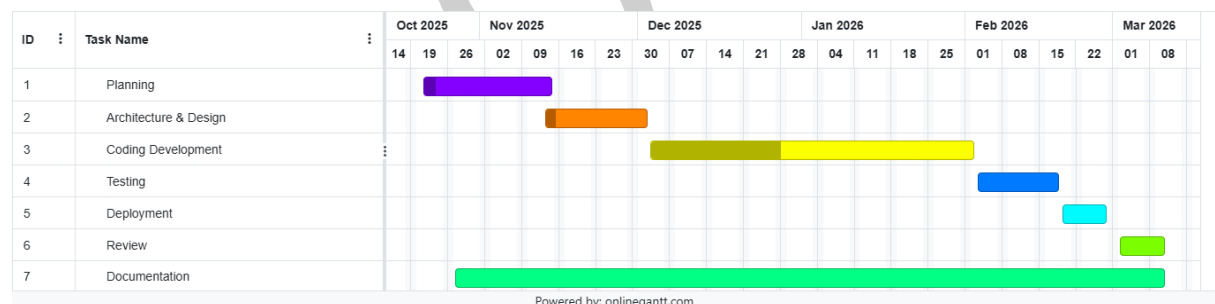


Figure 3. 3: Gantt Chart of BeemaSathi

3.1.3 Object modelling: Object & Class Diagram

The class diagram defines the 14 core models of BeemaSathi with their attributes, methods, and relationships. The object diagram provides a concrete runtime snapshot showing a specific client purchasing a policy and making a verified payment.

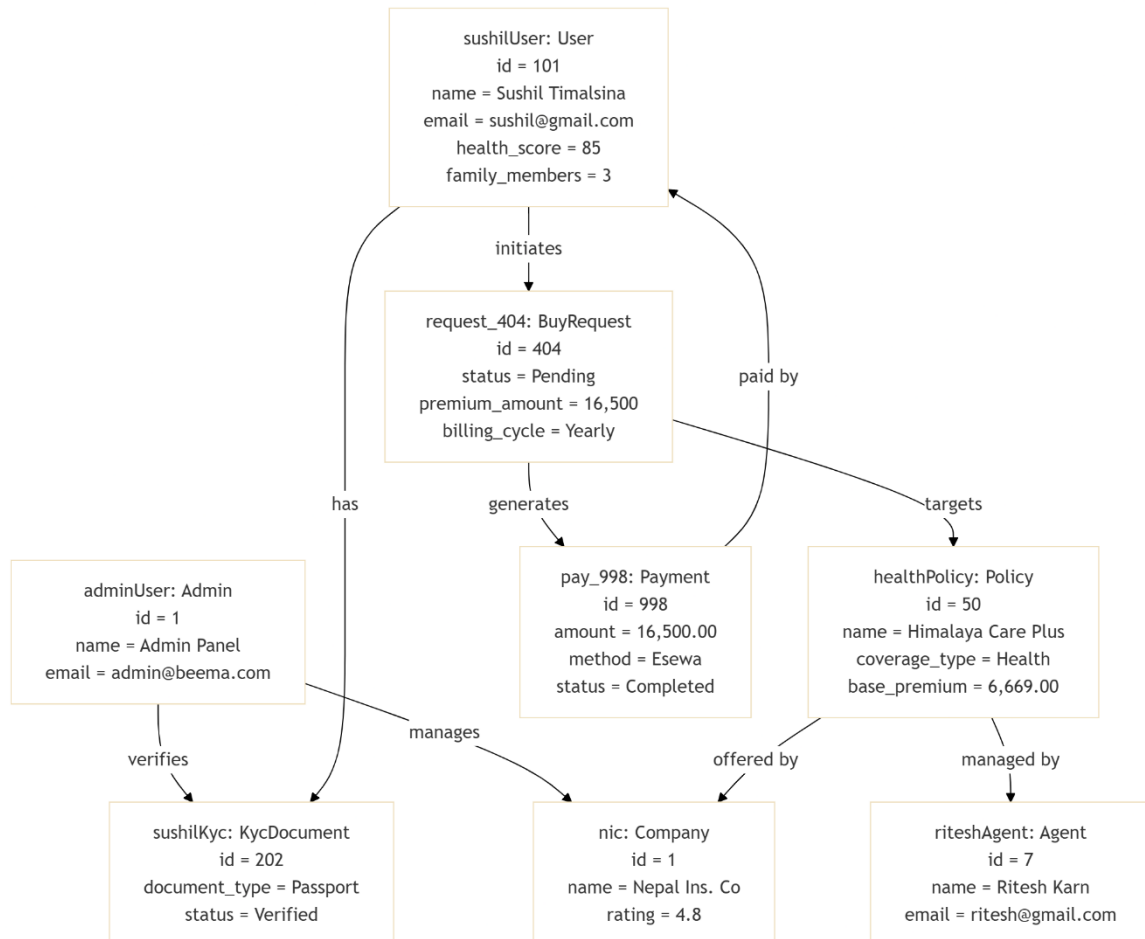


Figure 3. 4: Object Diagram of BeemaSathi

The object diagram represents a specific instance of the BeemaSathi system at runtime. It shows a registered client Sushil Timalsina with an approved KYC document submitting a buy request for the Himalayan Care Plus policy offered by Nepal Ins. Co. company. The buy request is linked to a yearly billing cycle with an approved status. Once the buy request is approved, Ram initiates a payment of Rs. 16,500 via eSewa which gets verified by the system. Each object instance directly corresponds to a class defined in the class diagram, illustrating how abstract class definitions translate into concrete data during actual system execution.

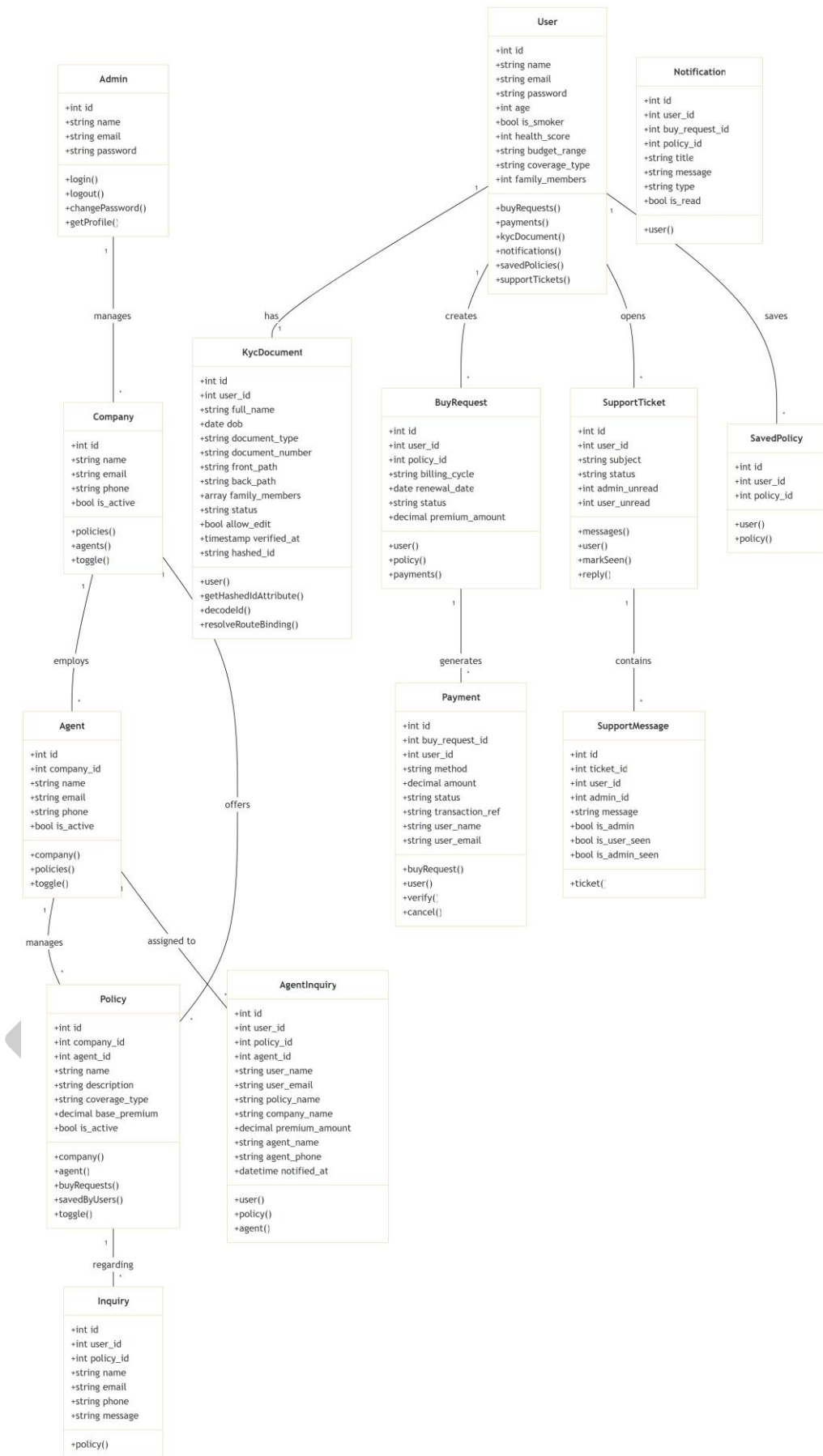


Figure 3. 5: Class Diagram of BeemaSathi

The class diagram represents the complete static structure of the BeemaSathi system, defining all 14 model classes along with their attributes, methods, and inter-class relationships.

The diagram is organized into four groups. The Administration group contains the Admin class which manages Company, Agent, and Policy classes through the obscured /htt/* routes. The Provider group contains Company, Agent, and Policy, where a Company employs multiple Agents and offers multiple Policies. The Client-side group contains User, KycDocument, SavedPolicy, SupportTicket, and SupportMessage, where a User has one KycDocument, raises multiple SupportTickets, and saves multiple Policies. The Transaction group contains BuyRequest, Payment, and Notification, where a User submits multiple BuyRequests, each generating multiple Payments, and each verified payment triggers a Notification.

Each class clearly lists its attributes with data types derived from the actual Laravel model files, including encoded ID fields for security, boolean flags, enum types, and timestamps. Methods listed represent Eloquent relationships and key business logic functions such as approve(), verify(), toggle(), and decodeId(). Multiplicities such as one-to-one, one-to-many, and many-to-many are shown on all relationship lines, providing a complete blueprint of the system's data layer.

3.1.4 Dynamic Modelling: State & Sequence diagram

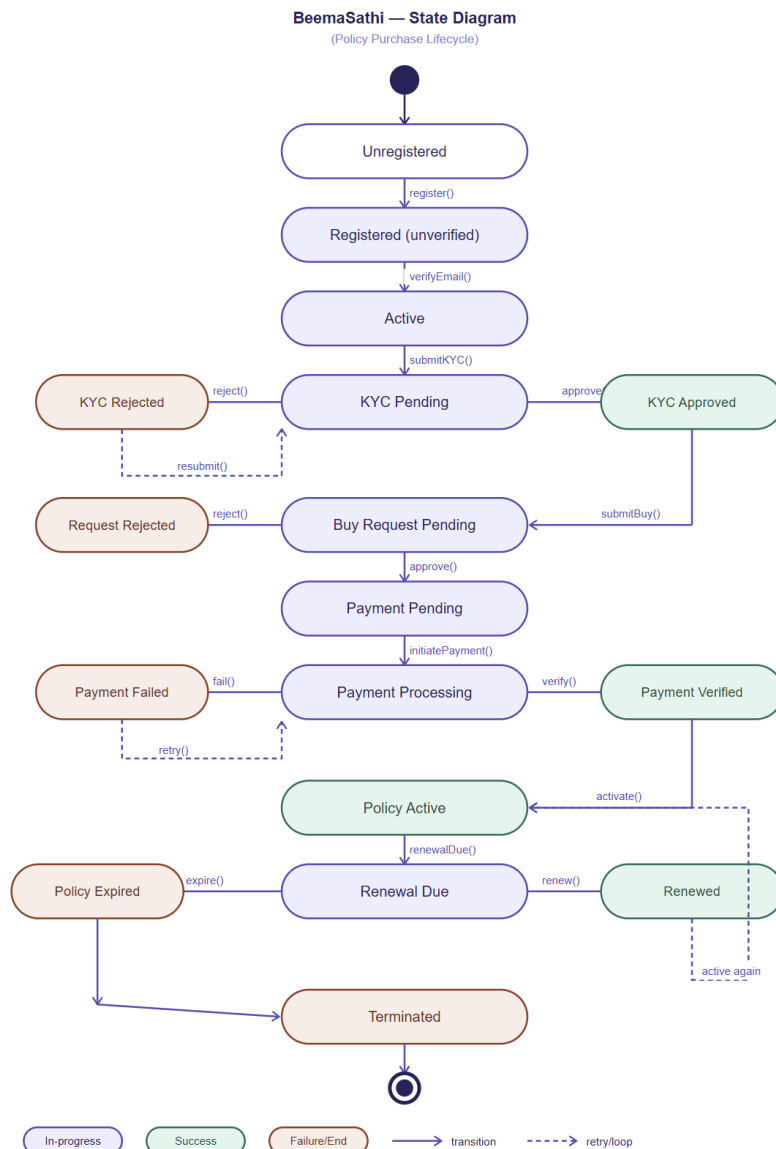


Figure 3. 6: State Diagram of Policy Purchase Lifecycle in BeemaSathi

The state diagram illustrates the complete lifecycle of a policy purchase in BeemaSathi, tracking how the system transitions through different states from the moment a user registers until the policy terminates.

The lifecycle begins with the Unregistered state, where a visitor has not yet created an account. Upon calling `register()`, the system moves to the Registered (unverified) state, waiting for email confirmation. Once `verifyEmail()` is triggered, the user enters the Active state and can interact with the platform fully.

From the Active state, the user submits KYC documents, moving the system into KYC Pending. The admin then reviews the submission. If rejected, the system enters the KYC Rejected state and the user must resubmit. If approved, the KYC record is locked and the system moves to KYC Approved, enabling the user to proceed with a purchase.

Upon submitting a buy request, the system enters Buy Request Pending. The admin reviews it and either rejects it, ending that request, or approves it, transitioning to Payment Pending. The user then initiates payment, moving the system into Payment Processing. If the payment gateway returns a failure, the system enters Payment Failed and allows a retry. Upon successful verification, the system transitions to Policy Active.

From Policy Active, the scheduled command monitors renewal dates. When a renewal becomes due, the system enters Renewal Due. If the user makes a renewal payment, the system transitions to Renewed and returns to Policy Active. If the renewal is not made within the grace period, the system moves to Policy Expired and finally to Terminated, which is the end state.

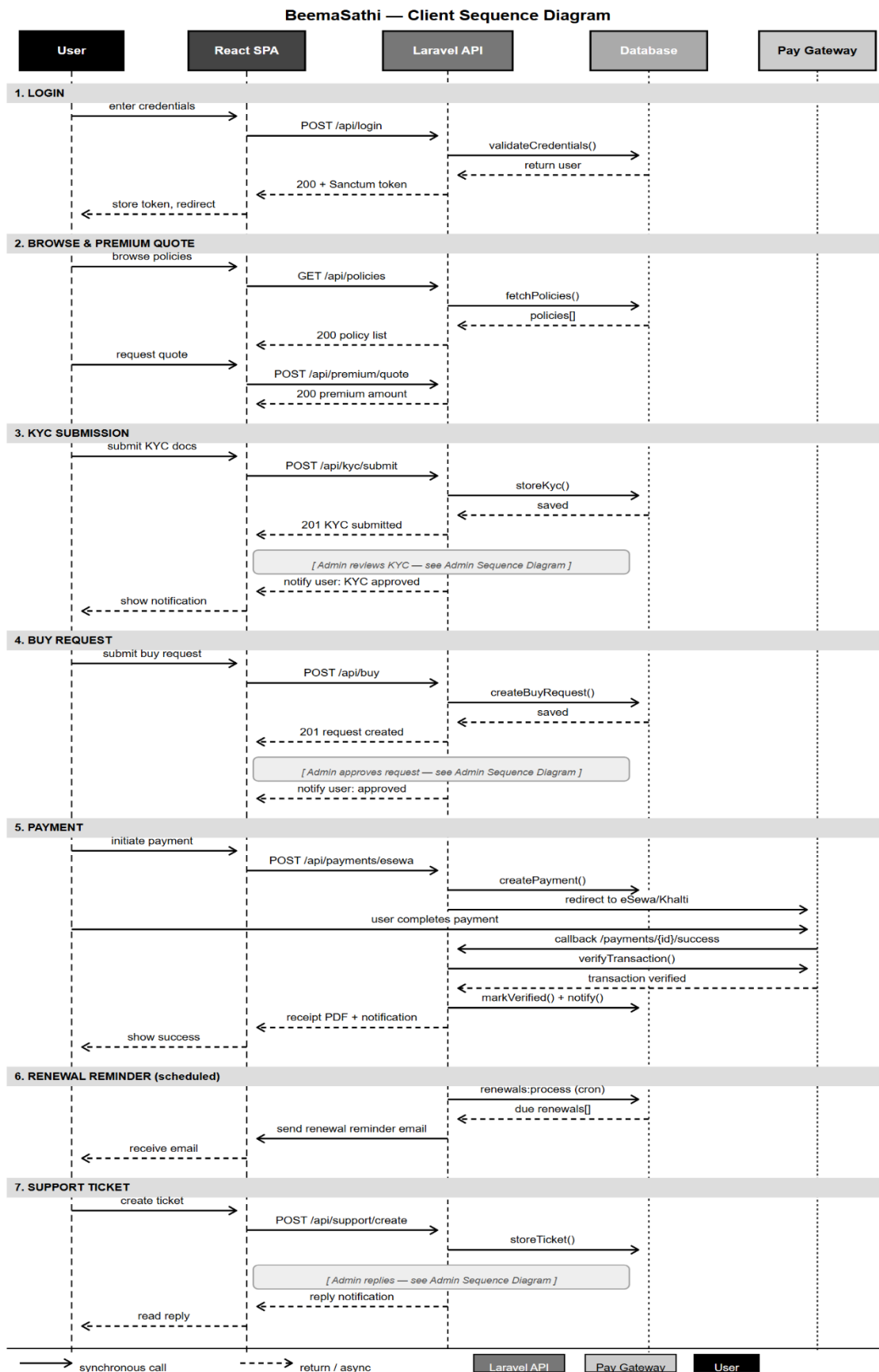


Figure 3. 7: Client Sequence Diagram of BeemaSathi

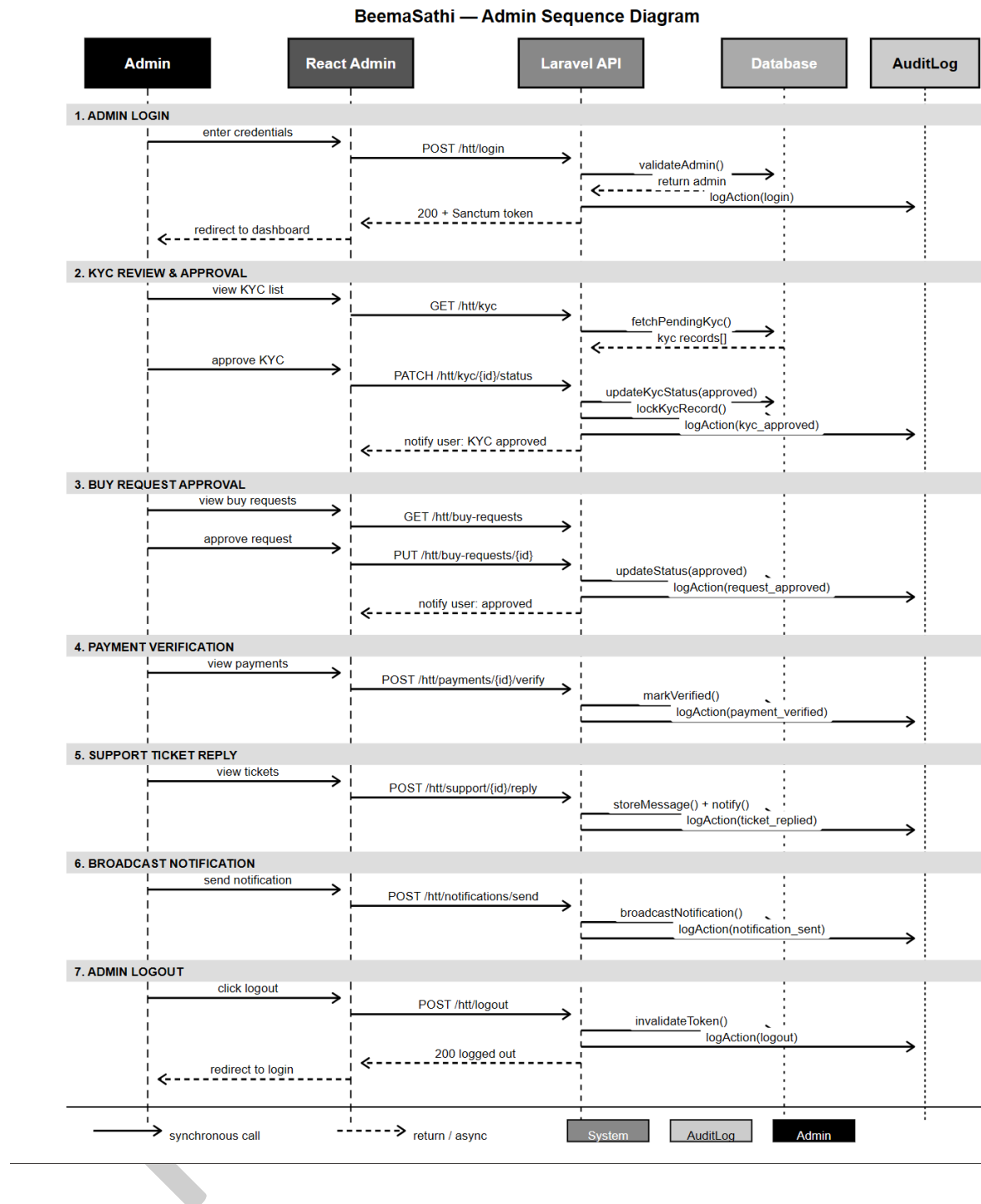


Figure 3. 8: Admin Sequence Diagram of BeemaSathi

The sequence diagram illustrates the dynamic interaction between six system participants User, React SPA, Laravel API, Database, Payment Gateway, and Admin across eight key flows.

In the Login flow, the user submits credentials through the React frontend, which sends a POST request to `/api/login`. The Laravel API validates the credentials against the database and returns a Sanctum token, which the frontend stores in `sessionStorage`.

In the Browse and Premium Quote flow, the React SPA fetches the policy list via GET `/api/policies`. The user then requests a premium quote via POST `/api/premium/quote`, and the API returns the calculated premium amount using the PremiumCalculator service.

In the KYC flow, the user submits documents via POST `/api/kyc/submit`. The API stores the KYC record in the database and the admin reviews it via PATCH `/http/kyc/{id}/status`. Upon approval, the system sends a notification to the user.

In the Buy Request flow, the user submits a purchase via POST `/api/buy`. The API creates a BuyRequest record. The admin then approves it via PUT `/http/buy-requests/{id}` and the system notifies the user of the approval.

In the Payment flow, the user initiates payment via POST `/api/payments/esewa` or `/api/payments/khalti`. The API redirects the user to the respective payment gateway. After the user completes payment, the gateway sends a callback to the success endpoint. The API verifies the transaction, marks the payment as verified, generates a receipt PDF, and sends a notification to the user.

In the Renewal flow, the `scheduled renewals:process` command queries the database for due renewals and automatically sends reminder emails to the respective clients.

In the Support Ticket flow, the user creates a ticket via POST `/api/support/create`. The admin replies via POST `/http/support/{id}/reply`. The system stores the message and notifies the user, who can then read the reply from the dashboard.

Finally, in the Logout flow, the user clicks logout, triggering POST `/api/logout`. The API invalidates the Sanctum token and the frontend redirects the user back to the home page.

3.1.5 Process modelling: Activity Diagram

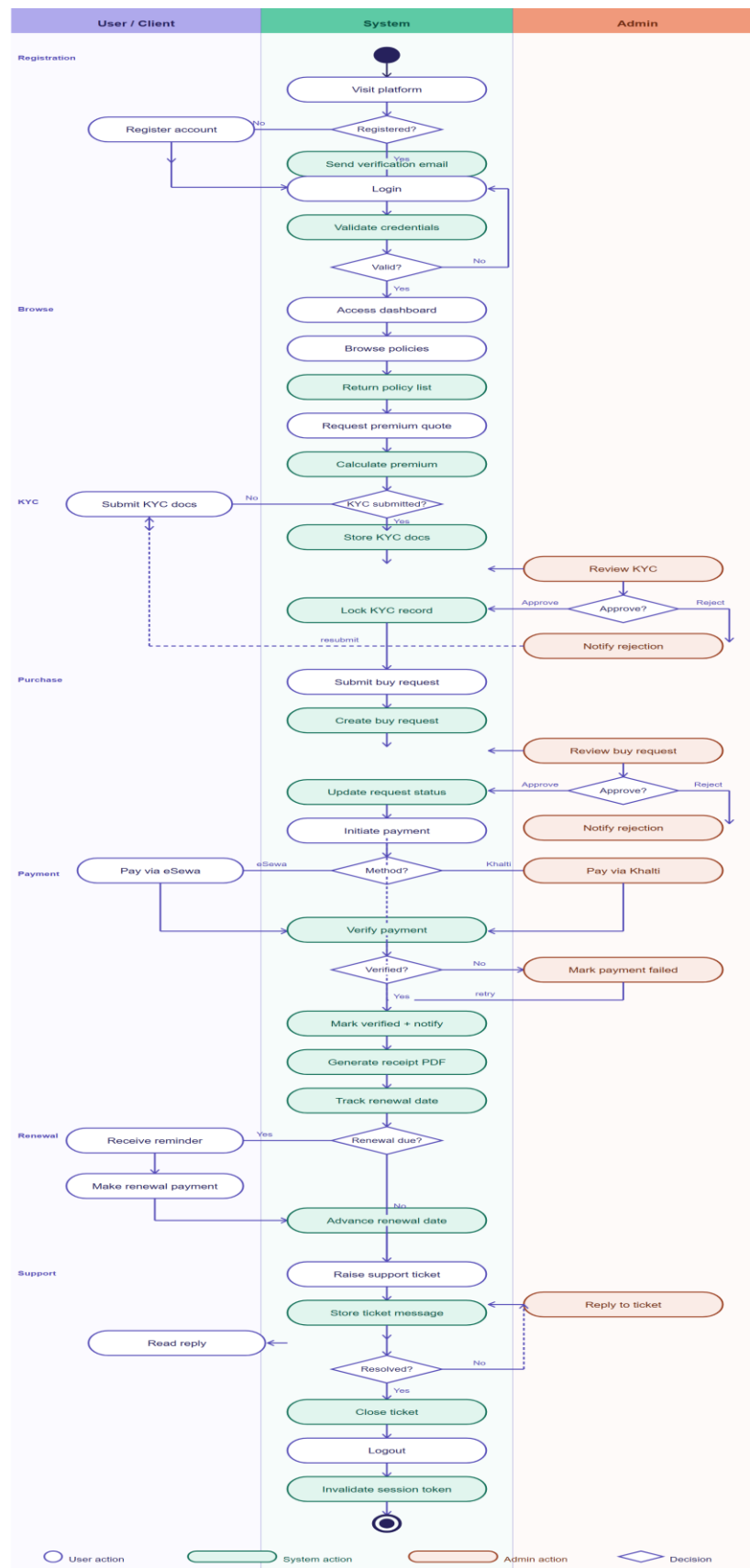


Figure 3. 9: Process modelling: Activity Diagram

The activity diagram illustrates the complete end-to-end flow of the BeemaSathi system across three swim lanes: User, System, and Admin, covering registration, login, policy browsing, premium calculation, KYC submission and approval, buy request processing, payment via eSewa or Khalti, renewal tracking, support ticket handling, and logout, with decision points and alternate paths shown throughout.

3.2 System Design

The System Design phase of BeemaSathi serves as the technical bridge between initial requirements and a functional platform, focusing on creating a scalable and maintainable architecture. By organizing the system into logical modules with efficient data flows, the design ensures high performance and reliability. This structured approach allows the platform to handle complex insurance processes while remaining simple enough for long-term technical management and updates.

A key aspect of this design is the decomposition of the system into independent components such as policy management, premium calculation, and payment handling. These modules operate autonomously but communicate through standardized interfaces, providing the flexibility needed to integrate future enhancements like new payment gateways or advanced AI recommendations. This modularity ensures that the system can grow in complexity without compromising its stability.

Security and architecture are further reinforced through a strict separation of the React frontend and Laravel backend, alongside a distributed deployment strategy. Sensitive operations like KYC processing and financial transactions are protected by secure API endpoints and structured database controls to maintain data integrity. By distributing the workload across specialized servers and external services, the design guarantees that BeemaSathi remains a secure, responsive, and robust digital insurance solution.

3.2.1 Refinement of Classes and Object

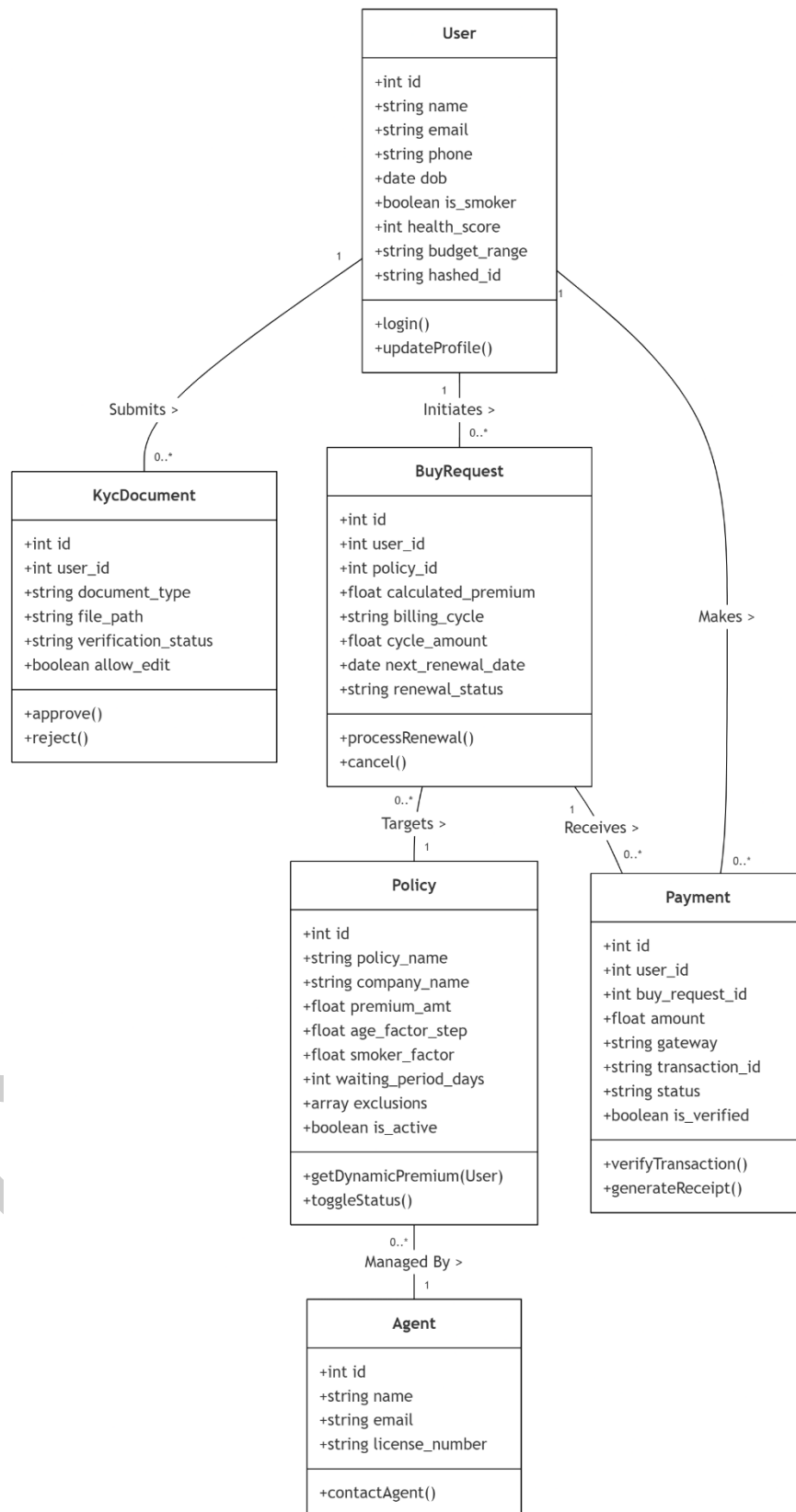


Figure 3. 10: Refinement of Classes for BeemaSathi

The class diagram presents the core structure of BeemaSathi with six primary classes and their relationships. The User class sits at the top, containing attributes such as id, name, email, dob, is_smoker, health_score, budget_range, and hashed_id, with methods login() and updateProfile(). A single User submits one KycDocument containing document type, file path, verification status, and allow_edit flag, with approve() and reject() methods.

The User also initiates multiple BuyRequest instances, each storing user_id, policy_id, calculated_premium, billing_cycle, cycle_amount, next_renewal_date, and renewal_status, with processRenewal() and cancel() methods. Each BuyRequest targets one Policy, which holds policy name, company name, premium amount, age and smoker factors, waiting period, exclusions, and active status, with getDynamicPremium() and toggleStatus() methods. Each Policy is managed by one Agent having name, email, and license number with a contactAgent() method.

The User also makes multiple Payment instances linked to a BuyRequest, storing amount, gateway, transaction ID, status, and is_verified flag, with verifyTransaction() and generateReceipt() methods.

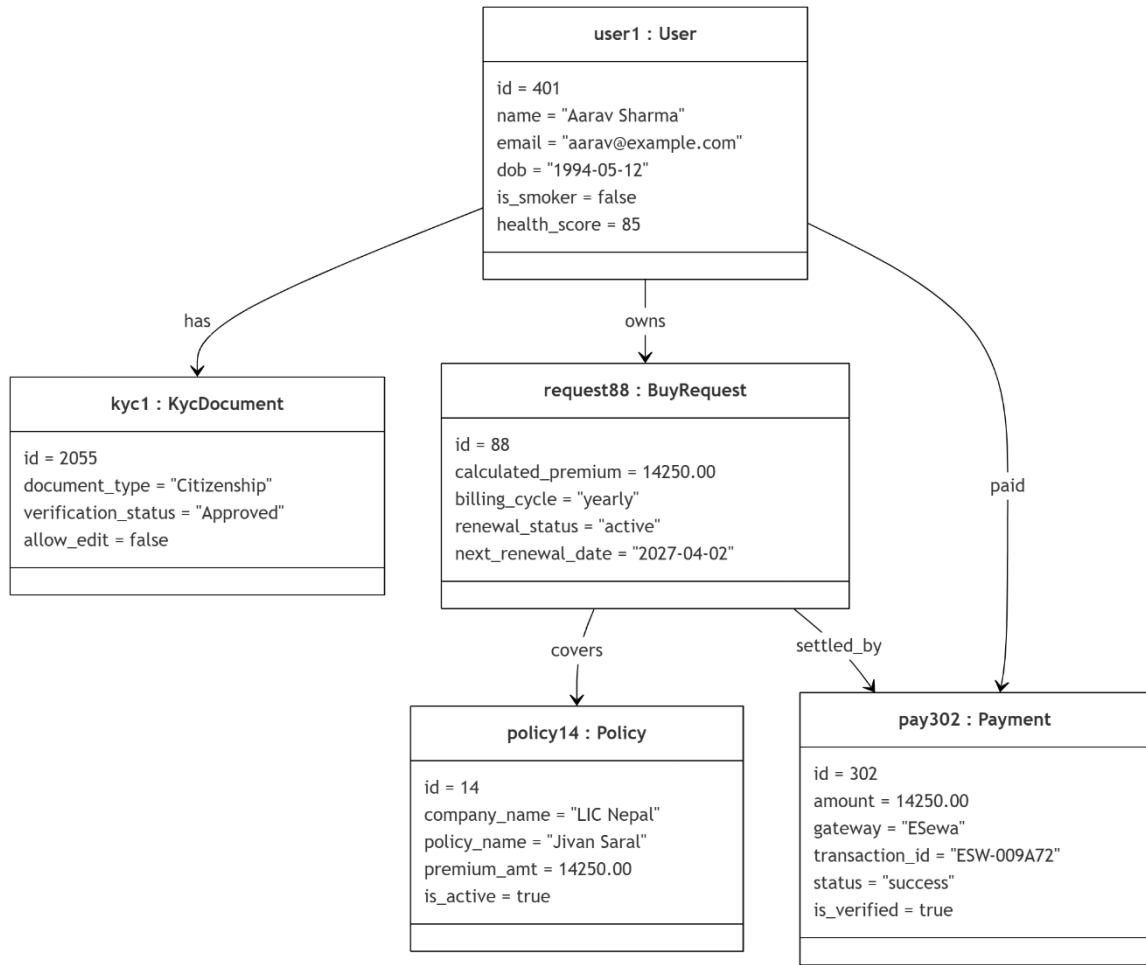


Figure 3. 11: Refinement of Object Diagram for BeemaSathi

The object diagram presents a concrete runtime instance of the BeemaSathi system. A specific user `user1 : User` with id 401, named Aarav Sharma, is a non-smoker with a health score of 85. This user has one approved `kyc1 : KycDocument` of type Citizenship with `allow_edit` set to false, meaning the record is locked after admin approval.

The same user owns `request88 : BuyRequest` with a calculated premium of Rs. 14,250, a yearly billing cycle, active renewal status, and next renewal date set to 2027-04-02. This buy request covers `policy14 : Policy` named Jivan Saral offered by LIC Nepal with a base premium of Rs. 14,250. The difference between the base premium and calculated premium reflects the dynamic computation performed by the PremiumCalculator service based on the user's age, health score, and smoking status.

The buy request is settled by `pay302 : Payment` with amount Rs. 14,250 paid via eSewa with transaction ID ESW-009A72, status success, and `is_verified` set to true. The same payment is also directly linked to the user through the paid relationship, confirming that the

user made the payment. This object diagram demonstrates how abstract class definitions from the class diagram translate into actual, meaningful data instances during system execution.

3.2.2 Component diagram

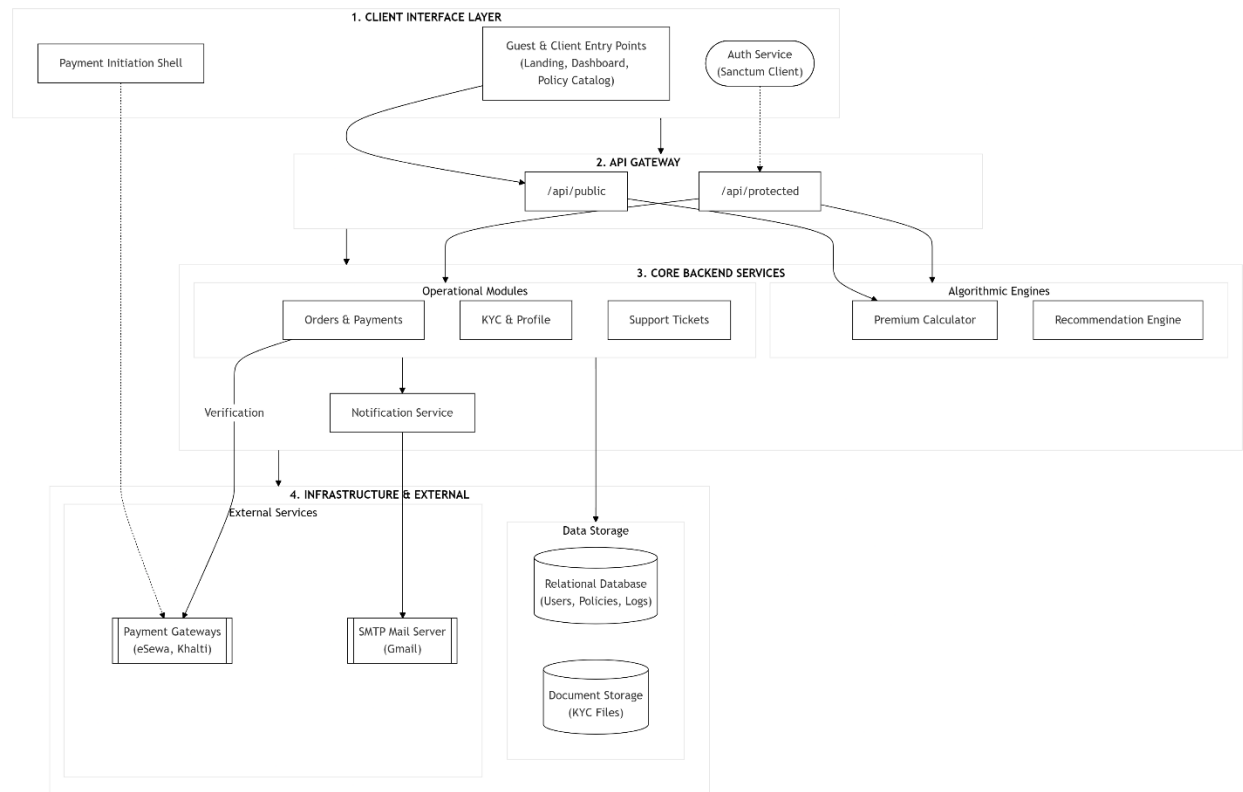


Figure 3. 12: Component Diagram of BeemaSathi for User

The component diagram shows BeemaSathi divided into two main subsystems with three external services.

The **React SPA Frontend** subsystem contains Public Pages, Auth Pages, Client Dashboard, Admin Dashboard, Axios API Client, Route Guards, and Auth Broadcast components. All communication to the backend passes through the Axios API Client via the REST API interface.

The **Laravel API Backend** subsystem contains Sanctum Auth, Route Controllers, AdminMiddleware, AuditLogMiddleware, PremiumCalculator, NotificationService, Renewal Command, Eloquent ORM, and Payment Handlers. The Eloquent ORM component queries the MySQL Database. The Payment Handlers component initiates transactions with eSewa RC v2 and Khalti API v2. The NotificationService dispatches emails through the SMTP Mail Server.

The three **external services** eSewa, Khalti, and the SMTP Mail Server sit outside both subsystems and are consumed by the backend layer only.



Figure 3. 13: Component Diagram of BeemaSathi for Admin

3.2.3 Deployment diagram

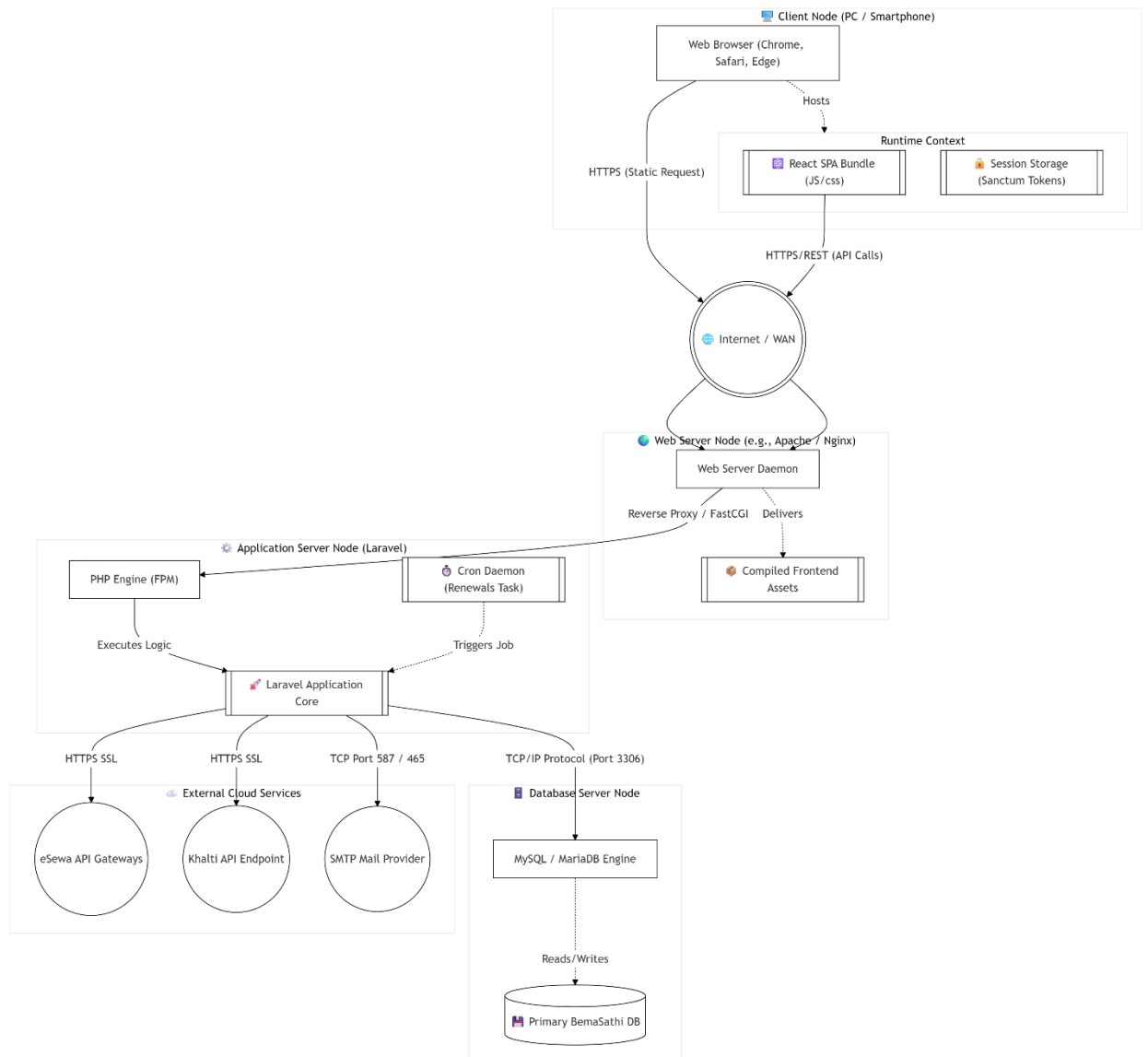


Figure 3. 14: Deployment Diagram for BeemaSathi

The deployment diagram shows how BeemaSathi is physically distributed across five nodes.

The **Client Browser** node runs the React SPA, stores the Sanctum token in sessionStorage, and uses BroadcastChannel for cross-tab auth sync. It communicates with the web server over HTTPS REST API calls.

The **Web/App Server** node hosts the Laravel API with all its artifacts Sanctum middleware, PremiumCalculator, NotificationService, and the renewals:process cron command. This is the central node that connects to all other nodes.

The **DB Server** node runs MySQL with all Laravel migrations applied, accessed by the web server over TCP/IP.

The **Mail Server** node receives outbound emails from NotificationService via SMTP for KYC updates, payment confirmations, and renewal reminders.

The two **External** nodes eSewa RC v2 and Khalti API v2 receive payment initiation requests from the web server over HTTPS and return transaction verification responses via callback URLs.

The **Admin Browser** node runs the same React SPA but accesses the system exclusively through the obscured /htt/* admin routes.

3.3 Algorithm

3.3.1 Adaptive Hybrid Policy Matcher (AHPM)

The recommendation algorithm is implemented in the BeemaSathi backend as a custom rule-based engine that suggests the most suitable insurance policies to a registered user based on their profile attributes. No external API or machine learning model is used. The algorithm is accessible through two endpoints `GET /api/recommendations` for general recommendations and `GET /api/recommendations/personal` for personalized recommendations based on the authenticated user's saved profile.

The algorithm works by scoring each active policy against the user's profile and ranking them in descending order of relevance. The scoring process considers the following factors:

Age Suitability The algorithm checks whether the user's age falls within the optimal coverage range for a given policy. Policies designed for younger users are ranked higher for younger clients and lower for older clients, and vice versa. This ensures that the recommended policies are age-appropriate and financially sensible for the user.

Smoking Status Match Policies with lower smoker risk factors are ranked higher for non-smoking users, while policies that explicitly accommodate smokers are ranked higher for users who have indicated smoking status on their profile. This prevents recommending policies with prohibitively high premiums to users who may not benefit from them.

Health Score Compatibility The user's health score is compared against the policy's risk tolerance. Policies that yield a lower computed premium for a high health score user are ranked higher, encouraging healthier users towards more cost-effective options.

Budget Range Alignment The algorithm computes the expected premium for each policy using the `PremiumCalculator` service and compares it against the user's stated budget range. Policies whose computed premiums fall within or close to the user's budget are ranked higher, while policies that significantly exceed the budget are ranked lower or excluded entirely.

Coverage Type Preference If the user has specified a preferred coverage type on their profile such as life, health, or comprehensive, policies matching that coverage type receive a higher relevance score. Policies of other coverage types receive a lower score but are not excluded, allowing users to discover options outside their stated preference.

Family Members Consideration If the user has indicated a number of family members on their profile, policies that support family coverage are ranked higher. Single-coverage policies are ranked lower for users with family members, as they would not provide adequate protection for the user's stated needs.

The final recommendation list is assembled by sorting all active policies by their computed relevance score in descending order. The top-ranked policies are returned to the frontend, where they are displayed on the client dashboard as personalized suggestions. The personal recommendation endpoint additionally filters results using the authenticated user's approved KYC data where available, further refining the output based on verified identity information.

This rule-based approach ensures that recommendations are fully explainable, consistent, and adjustable by tuning the scoring weights without requiring any model training or external dependency.

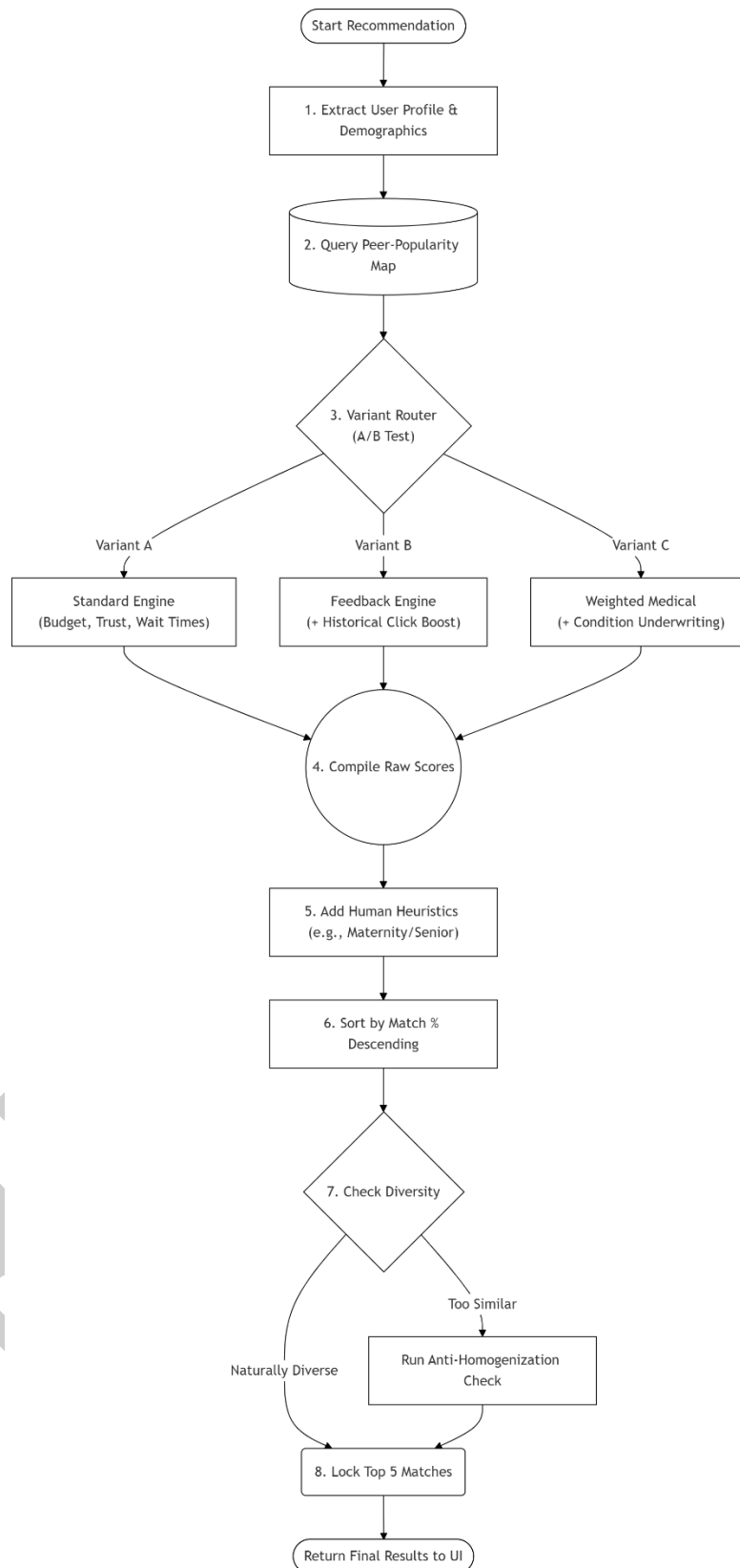


Figure 3. 15: Adaptive Hybrid Policy Matcher (AHPM) Diagram

3.3.2 Dynamic Actuarial Pricing Engine (DAPE) Algorithm

The **Dynamic Actuarial Pricing Engine (DAPE)** is the core deterministic rules-based premium calculation algorithm implemented in the PremiumCalculator service class. It computes a personalized insurance premium for a given user and policy combination based on several weighted profile factors.

The algorithm begins with the base premium defined on the selected policy. It then applies a series of multipliers based on the following inputs:

Age Factor The user's age is derived from their date of birth or directly from their profile. Younger users receive a lower age multiplier, reflecting lower actuarial risk, while older users receive a progressively higher multiplier.

Smoker Factor If the user is marked as a smoker, a fixed risk multiplier is added to the premium to account for the elevated health risk associated with smoking.

Health Score Factor The user's health score, ranging from a low to high scale, inversely affects the premium. A higher health score reduces the computed premium while a lower health score increases it.

Coverage Type Factor Different coverage types such as individual, family, or comprehensive carry different base multipliers that adjust the final premium amount accordingly.

Family Members Factor If the user includes additional family members under the policy, each additional member adds a proportional increment to the computed premium.

Budget Range Factor The user's stated budget range is used to validate and optionally adjust the computed premium to ensure it falls within an acceptable range.

If any of these inputs are not explicitly provided in the request, the algorithm falls back to the authenticated user's saved profile fields and their approved KYC data. The final computed premium is returned as a quoted amount through the PremiumQuoteController via POST /api/premium/quote and also through POST /api/premium/calculate.

The algorithm can be expressed as:

Final Premium = Base Premium

× Age Factor

× Smoker Factor

× (1 - Health Score Adjustment)

× Coverage Type Multiplier

+ (Family Members × Per Member Rate)

This rule-based approach ensures that premium computation is fully transparent, predictable, auditable, and easily adjustable by modifying the factor weights without requiring model retraining.

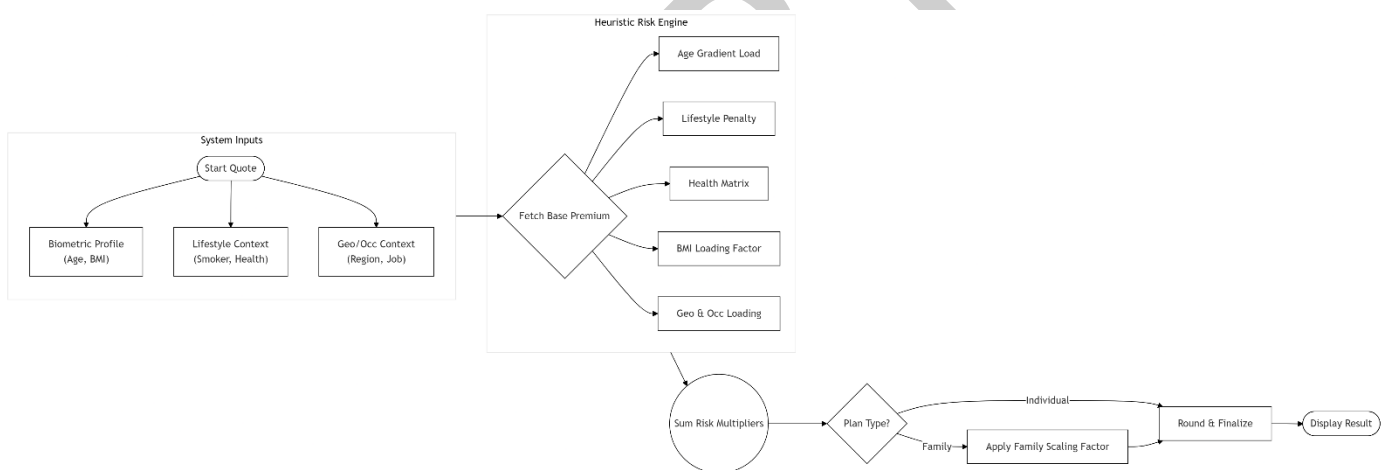


Figure 3. 16: Dynamic Actuarial Pricing Engine (DAPE) Algorithm Diagram

Chapter 4: Implementation and Testing

4.1 Implementation

The Implementation phase of BeemaSathi involves developing the system using React for the frontend and Laravel for the backend. Core features such as authentication, policy management, premium calculation, and payment integration are implemented using a modular approach. RESTful APIs enable communication between components, while MySQL handles data storage. Security measures ensure safe and reliable system operation.

4.1.1 Tools Used (CASE Tools, Programming Language, Database Platforms)

The development of BeemaSathi involved a carefully selected set of tools and technologies chosen for their reliability, compatibility, and suitability for building a full-stack insurance marketplace platform.

Frontend Tools

React 19 is the primary frontend framework used to build the BeemaSathi client interface as a Single Page Application. It provides a component-based architecture that allows reusable UI elements across the client dashboard and admin console. React Router handles client-side navigation between public, client, and admin pages without full page reloads.

Vite is used as the frontend build tool and development server. It provides extremely fast hot module replacement during development and optimized production builds through efficient bundling.

Tailwind CSS is used for styling the entire frontend interface. Its utility-first approach allows rapid and consistent design implementation without writing custom CSS files, ensuring a responsive and clean user interface across all device sizes.

Axios is used as the HTTP client for making API requests from the frontend to the Laravel backend. It handles request headers, token attachment, and response interception for both the client and admin API clients.

Recharts is used for rendering data visualizations on the admin dashboard, including payment statistics and policy performance charts. Framer Motion is used for smooth UI animations and page transition effects.

Backend Tools

Laravel is the primary backend framework used to build the BeemaSathi REST API. It provides a structured MVC architecture, built-in routing, middleware support, Eloquent ORM for database interaction, scheduled commands for renewal processing, and Blade templating for PDF generation.

Laravel Sanctum is used for API token-based authentication. It issues tokens on login, protects all authenticated routes, and allows separate authentication flows for clients and administrators.

Eloquent ORM is used for all database interactions. It maps database tables to PHP model classes and handles relationships, query building, and migrations in a clean and expressive way.

PHP is the server-side language powering the Laravel application. It handles all business logic including premium calculation, payment verification, KYC processing, and notification dispatch.

Database

MySQL is used as the relational database management system. It stores all application data including users, policies, buy requests, payments, KYC documents, notifications, support tickets, and audit logs. Laravel migrations manage the database schema versioning.

Payment Gateways

eSewa RC v2 is integrated for digital wallet payments. The backend creates a payment payload, redirects the user to the eSewa checkout page, and verifies the transaction through a success callback endpoint.

Khalti API v2 is integrated as the second payment option. The backend initiates a payment via the Khalti API, receives a payment URL, and verifies the transaction through a return URL lookup after the user completes payment.

Development Tools

XAMPP is used as the local development environment. It provides Apache, MySQL, and PHP in an integrated package, allowing the Laravel backend to run locally without requiring a remote server during development.

Git is used for version control, tracking code changes across both the frontend and backend repositories throughout the development lifecycle.

Thunder Client is used for testing and debugging all backend API endpoints during development, verifying request and response structures before frontend integration.

Visual Studio Code is used as the primary code editor for both frontend and backend development, with extensions for PHP, JavaScript, Tailwind CSS, and Laravel support.

4.1.2 Implementation details of modules

The BeemaSathi platform is organized into several functional modules, each responsible for a specific set of features. The following describes the implementation details of each major module.

Authentication Module

The authentication module handles registration, login, and logout for both clients and administrators through separate flows. Client registration accepts name, email, and password, creates a user record, and sends a verification email. Login validates credentials and issues a Laravel Sanctum token which the frontend stores in `sessionStorage`. The admin login follows the same token-based approach but targets the `Admin` model separately and is accessible only through the obscured `/http/login` endpoint. Route guards `ProtectedClientRoute` and `ProtectedAdminRoute` on the frontend prevent unauthorized access to protected pages. An idle logout hook `useIdleLogout` automatically invalidates the admin session after a period of inactivity. Cross-tab authentication state is synchronized using the BroadcastChannel API through `authBroadcast.js`.

Policy Management Module

The policy management module allows public users to browse all active insurance policies and view individual policy details without authentication. Administrators can create, update, delete, and toggle the active status of policies through the `/http/policies` endpoints. Each policy is linked to a company and an agent. Primary keys are obfuscated on frontend URLs using a base64 and XOR hashed ID strategy to prevent ID enumeration.

Premium Calculation Module

The premium calculation module exposes two endpoints `/api/premium/calculate` and `/api/premium/quote` handled by dedicated controllers. The `PremiumCalculator` service computes a personalized premium based on the user's age or date of birth, smoking status, health score, coverage type, budget range, and number of family members. Each factor applies a weighted multiplier to the base premium defined on the selected policy. If optional inputs are not provided, the service falls back to the authenticated user's profile and approved KYC data. This module is entirely rule-based with no external API or machine learning dependency.

KYC Module

The KYC module allows registered users to submit their identity documents and personal details including full name, date of birth, address, province, district, municipality, document type, document number, and front and back document images. The submission is stored with a pending status. Administrators review submissions through the `/http/kyc` endpoints and either approve or reject them. Approved KYC records are locked by setting `allow_edit` to false, preventing unauthorized modifications. Administrators can grant edit access to a specific user when corrections are required. The `KycDocument` model uses a custom `hashed_id` attribute and overrides route model binding to support both numeric and encoded ID resolution.

Buy Request Module

The buy request module allows KYC-approved users to submit a policy purchase request through `POST /api/buy`. The system creates a `BuyRequest` record storing the user ID, policy ID, calculated premium, billing cycle, and initial renewal date. Administrators

review pending requests through the `/http/buy-requests` endpoints and update their status to approved or rejected. Upon approval, the system sends a notification to the client informing them to proceed with payment. Users can also preview their buy request details before final submission through `POST /api/buy/preview`.

Payment Module

The payment module supports two Nepali payment gateways eSewa and Khalti. For eSewa, the backend creates a `Payment` record and returns a signed payload that the frontend uses to redirect the user to the eSewa RC v2 checkout page. Upon successful payment, eSewa sends a callback to `/api/payments/{payment}/success` where the backend verifies the transaction status and marks the payment as verified. For Khalti, the backend calls the Khalti API v2 to initiate a payment and returns a `payment_url` to the frontend. After the user completes payment on the Khalti interface, the return URL triggers verification through a lookup call. The renewal logic distinguishes between the first verified payment as the initial purchase and subsequent payments as renewals, advancing the `next_renewal_date` accordingly. A personalized receipt PDF is generated using a Blade template that greets the client by name and displays their email address, available for download by both the client and the administrator.

Renewal Management Module

The renewal management module is implemented as a scheduled Laravel command `renewals:process` that runs automatically via the task scheduler. It queries all buy requests with renewal dates that are due or past due, marks them accordingly, and dispatches reminder email notifications to the respective clients through the `NotificationService`. Administrators can also manually trigger renewal reminders for individual buy requests through `POST /http/renewals/{buyRequest}/notify`. The admin renewals page provides a full list of all renewal records with their current status.

Notification Module

The notification module stores system-generated and admin-broadcast notifications in the `notifications` table. Notifications are created automatically on key events such as KYC approval or rejection, buy request approval, and payment verification. The `NotificationService` handles both in-app notification storage and optional email

delivery via SMTP. Administrators can also broadcast custom notifications to all users through `POST /http/notifications/send`. Clients can view all their notifications from the dashboard.

Support Ticket Module

The support ticket module allows clients to create tickets with a subject and message through `POST /api/support/create`. Both the client and administrator can exchange messages within a ticket thread. Read and unread states are tracked independently for each party using `admin_unread` and `user_unread` counters. Administrators reply through `/http/support/{ticket}/reply` and can update the ticket status through `/http/support/{ticket}/status`. The client can mark messages as seen through `POST /api/support/{ticket}/mark-seen`. All communication is centralized within the platform without relying on external messaging tools.

Admin Console Module

The admin console module provides a comprehensive management interface accessible exclusively through the `/http/*` route prefix. It covers policies, agents, companies, clients, KYC approvals, buy requests, payments, renewals, notifications, reports, and audit logs. Every admin action is automatically recorded by `AuditLogMiddleware` into the `audit_logs` table, creating a complete audit trail. Administrators can export audit logs and generate system reports through dedicated export endpoints. The admin profile page includes a secure modal-based password rotation flow through `POST /http/profile/change-password`. An idle logout mechanism on the admin frontend automatically terminates the session after inactivity to maintain security.

Agent and Company Module

The agent and company module allows administrators to manage insurance providers and their assigned agents. Companies are created with name, email, phone, and active status. Agents are created with name, email, phone, and a company assignment. Agent passwords are auto-generated securely by the backend upon creation, removing the need for manual password entry. Both agents and companies can be toggled active or inactive through dedicated toggle endpoints, which controls their visibility across the platform.

4.2 Testing

System testing for BeemaSathi is conducted to ensure that the integration between the React frontend and Laravel backend functions correctly, particularly regarding sensitive financial transactions and private user data. The testing process focuses on validating the custom rule-based algorithms for premium calculation and the security of the payment gateway integrations.

4.2.1 Test case for Unit Testing

In unit testing, the **BeemaSathi** system is broken down into its smallest functional components to ensure each part operates correctly in isolation. This modular approach is critical for verifying the integrity of the custom insurance logic and secure API interactions.

User registration

Table 4. 1: Test Case for User Registration of BeemaSathi

S.N	Action	Input	Expected Output	Actual Output	Test Result
1	Open application	http://localhost:5173/	Welcome page	Welcome page	Pass
2	User Register	Navigate to Get started	Redirect to Register page	Open register page	Pass
3	Enter invalid Email format	-Name: Sushil Timalina - Email: sushiltimal.com	Frontend validation error Enter a valid email address.	Registration failed	Pass
4	Password Mismatch	- Password: Password@123 - Confirm: Password@321	Failed with error message	Failed with error message	Pass

5	Submit valid data	- Name: Hari Sharma - Email: sushil.timalsina13@gmail.com - Password asdfghjkA@1234567 -Confirm Password: asdfghjkA@1234567 Date of birth: 21/03/2005 Coverage Type: For Myself(Individual)	Redirect to Email Verification page showing "Check your email!" message.	Redirected to Email Verification page showing "Check your email!" message	Pass
6	Verify Displayed Email	N/A	Correct user email(e.g., sushil.timalsina13@gmail.com) shown in text.	Correct email displayed	Pass
7	Click Didn't get the email? Resend	Click " Didn't get the email? Resend" button	Trigger NotificationService to send a new SMTP verification link	New email triggered	Pass
8	Navigate to Login	Go to Login	Redirects to login Page	Redirected to login Page	Pass
9	Register through Google Account	Click on Continue with Google Enter Date of Birth & Coverage Type	Redirect to Google Auth Page & Email Verification page	Redirected to Google Auth Page & Email Verification page	Pass

User Login

Table 4. 2: Test Case for User Login of BeemaSathi

S.N	Action	Input	Expected Output	Actual Output	Test Result
1	Open LoginPage	http://localhost:5173/login	Display Login card with Google and Email options	Login page opened	Pass
2	Google OAuth Integration	Click “Continue with Google”	Redirect to Google Oauth consent screen	Redirected to Google	Pass
3	Invalid Login Attempt	Email: sushil.timalsina13@gmail.com Wrong Password	Backend returns “Invalid password.”	Login failed	Pass
4	Password Visibility	Click “Eye” icon in Password field	Toggle between masked (dots) and plain text password	Password masked/ unmasked	Pass
5	Valid Standard Login	Email: sushil.timalsina13@gmail.com Correct Password	Redirect to “Dashboard”; Navbar updates to show “Compare” and “All Policies	User Dashboard opened	Pass

6	Navigation Post-Login	Click “All Policies”	Redirect to full policy marketplace	Marketplace loaded	Pass
7	Forgot Password	Click “Forgot your password?”	Redirect to password reset request page	Reset page opened	Pass

Admin module

Table 4. 3:Test Case for Admin Module of BeemaSathi

S.N	Action	Input	Expected Output	Actual Output	Test Result
1	Open application	http://localhost:5173/	Public welcome page	Welcome page	Pass
2	Secure Admin Login Access	Navigate to http://localhost:5173/htt/login	Admin- specific login card appears; URL obfuscated from standard /admin	Admin login page opened	Pass
3	Enter invalid Admin Email	Email: admin@wrong.com Password: admin1234	Invalid email.	Login failed	Pass
4	Valid Admin Login	Email: admin@example.com Password: ChangeMe123	Redirect to /htt/dashboard; Sidebar shows management modules	Admin Dashboard opened	Pass
5	Verify Dashboard Stats	View system activity overview	Displays Total Users (10), Active Policies (50), and	Live data matched DB stats	Pass

			Revenue (1,63,503.63)		
6	Policy Management	Click “Policies” in sidebar	Click “Policies” in sidebar	Policy list opened	Pass
7	Agent Management	Click “Agents” in sidebar	UI for managing insurance agents and company assignments	Agent console opened	Pass
8	Recent Payments Check	View “Recent Payments” section	Shows all recent transactions (e.g., MetLife Health Global – 1,368.66)	Transaction history visible	Pass
9	Audit Log Tracking	Click “Audit Log” in sidebar.	Display history of sensitive admin actions via AuditLogMiddleware.”	Audit history opened	Pass
10	Upcoming Renewals	View “Upcoming Renewals” card	Lists policies due/upcoming (e.g., Shikhar Medi Protect – 07/01/2026)	Renewal tracking active	Pass
11	Guest Messages	Click “Guest Messages”	View inquiries from ContactUs.jsx with	Message list opened	Pass

			mailto:/tel: actions		
12	Admin Profile	Click Profile icon/Settings	Access modal-based password rotation flow.	Profile settings opened	Pass
13	KYC Review	- Select User -> View Documents -> Click “Approve”	KYC status updates to “Approved”; User is notified	Status updated successfully	Pass
14	Policy Toggle	Select Policy -> Click “Toggle Active	Policy status flips (Active/Inactive) in real-time	Status changed in marketplace	Pass
15	Audit Verification	Perform an action -> Check Audit Log	System records the admin’s IP, ID, and action performed	Log entry created correctly	Pass
16	KYC Status Update	PATCH /htt/kyc/{id}/status to ‘Approved’	Approved KYC is locked; allow_edit set to false unless granted	KYC approved and locked	Pass
17	Agent-Company Assignment	Select Agent -> Choose Company from dropdown	Relationship stored in Agent model; agent now linked to specific policies	Agent linked successfully	Pass
18	Admin Logout	Click “Logout”	Clear token; redirect to admin login	Logout successful	Pass

			page; session cleared in all tabs		
--	--	--	-----------------------------------	--	--

4.2.2 Test case for System Testing

System testing involves testing the complete system to ensure it meets the specified requirements and functions correctly. Below are some of the test cases used for system testing.

System testing

Table 4. 4: Test Case for System Testing of Beema Sathi

S.N	Test case Description	Tested data	Expected result	Actual Result	Test result
1	Personalized Quote Generation	User completes profile (Age, Smoker status, Health score)	POST /api/premium/calculate returns tailored rates; UI updates “Quick Snapshot”	Quotes adjusted based on profile factors	Pass
2	Premium Calculation Accuracy	Update Profile (Age/Smoker status) -> Check Quotes	PremiumCalculator updates all available quotes based on new profile factors	Prices adjusted correctly in the marketplace	
3	Profile-to-Purchase Continuity	Click “Buy” on a personalized quote	Profile data automatically maps to KYC Page to submit KYC Documents for Verification fields.	Redirects to KYC Page	Pass
4	KYC Submission (Client)	Upload Document + Personal Details	Data stored in kyc_documents; Status set to “Pending”	KYC submitted; status updated	Pass

5	KYC Field Lockdown	Attempt to edit approved KYC fields	Fields become read-only; allow_edit set to false.	Inputs locked after Approval	Pass
6	KYC Rejection/Re-edit	Admin sets “Allow Edit and sends remarks for rejected KYC	Frontend unlocks form fields for client to re-upload documents	Form unlocked for corrections	Pass
7	KYC-Buy Dependency	KYC-Buy Dependency	Attempt POST /api/buy with unapproved KYC	Backend blocks request; redirects to KYC Page requiring approval	Pass
8	Automated Renewal Processing	Execute renewals:process scheduled command	System identifies due/expired policies and triggers notifications/emails	Notifications sent to relevant clients	Pass
9	Final Purchase & Receipt	Approved KYC -> Pay via eSewa/Khalti	BuyRequest updates to “Paid”; Receipt PDF greets user by name	Transaction completed; PDF generated	Pass
10	Multi-Payment Lifecycle	Process a second verified payment for an existing BuyRequest	First payment marks “Initial Purchase”; subsequent marks “Renewal” and advances date.	Renewal date advanced after second payment	Pass
11	Support Ticket Synchronization	Client replies to ticket -> Admin views in /htt/support	Read/Unread states update; messages appear in real-time for both roles	Communication synced across layouts	Pass
12	Guest Inquiry Workflow	Submit form in ContactUs.jsx	Record stored in inquiries table; visible	Message appeared in	Pass

		with valid Policy link	in Admin “Guest Messages”	Admin panel with mailto: link	
13	ID Enumeration Prevention	Manually change encoded_id in URL to a random string	System fails to decode; returns 404 or redirects to “Policy Not Found”	Unauthorized ID access blocked	Pass
14	Cross-Tab Sync	Login/Action in Tab A; verify Tab B	authBroadcast.js keeps Sanctum tokens in sync across sessions.	Tabs synchronized correctly	Pass
15	Session Idle Timeout	Leave dashboard inactive.	useIdleLogout clears sessionStorage and redirects to login	Automated logout successful	Pass
16	Standard Logout	Click “Logout” in Client Profile Section	Token revoked; User redirected to Welcome page; sessionStorage cleared	Logged out and redirected to login page.	Pass
17	Multi-Tab Logout	Open Dashboard in two tabs; Logout in Tab A	authBroadcast.js triggers; Tab B automatically redirects to Welcome page	Both tabs logged out	Pass
18	Unauthorized Access	Click “Back” button after logout	React Router guards detect missing token and redirect to Login	Access blocked	Pass

Chapter 5: Conclusion and Future Recommendations

5.1 Conclusion

The **BeemaSathi** project successfully delivers a comprehensive, full-stack insurance marketplace tailored for the Nepalese digital landscape. By integrating a **Laravel Sanctum** backend with a **React** frontend, the system provides a seamless and secure environment for clients to browse, compare, and purchase insurance policies.

The technical implementation addresses key pain points in traditional insurance processing through automated, rule-based **Premium Calculations** and digitized **KYC workflows**. Security is a foundational element of the platform, achieved through **ID obfuscation**, **Audit logging**, and a secure **Admin console** accessed via the `/htt/` prefix to ensure management obscurity. With successful integrations of local payment gateways like **eSewa** and **Khalti**, BeemaSathi stands as a robust, SaaS-ready solution that enhances transparency and efficiency in the insurance sector.

5.2 Lesson learnt/Outcome

Developing this platform provided deep practical insights into modern software engineering practices:

- **Decoupled Architecture:** Gained proficiency in building and securing RESTful APIs that communicate with a Vite-powered React Single Page Application (SPA).
- **Security Implementation:** Learned to apply "Security through Obscurity" by refactoring sensitive routes and implementing middleware for audit tracking.
- **Financial Integration:** Successfully managed complex payment lifecycles, distinguishing between initial purchases and automated renewal logic.
- **State & Session Management:** Implemented cross-tab synchronization using `authBroadcast.js` and automated idle logouts to protect user data.

5.3 Future Recommendations

To further evolve **BeemaSathi** into a market-leading platform, the following enhancements are recommended:

- **Mobile Application:** Development of a native mobile app (Flutter or React Native) to provide real-time push notifications for policy renewals and easier document scanning for KYC.
- **Separate Login System for Company and Agent:** Implementing separate login system for Company and Agent helps to reduce burden on Admin for managing everything and helps to upgrade Insurance Marketplace to next level.
- **AI-Driven Personalization:** Transitioning from rule-based algorithms to Machine Learning models for more advanced, personalized policy recommendations based on user behavior.
- **OCR Integration:** Implementing Optical Character Recognition (OCR) to automatically extract data from uploaded citizenship documents, further streamlining the KYC process.
- **Agent Commission Tracking:** Expanding the Admin console to include detailed reporting and automated commission disbursements for agents linked to successful buy requests.

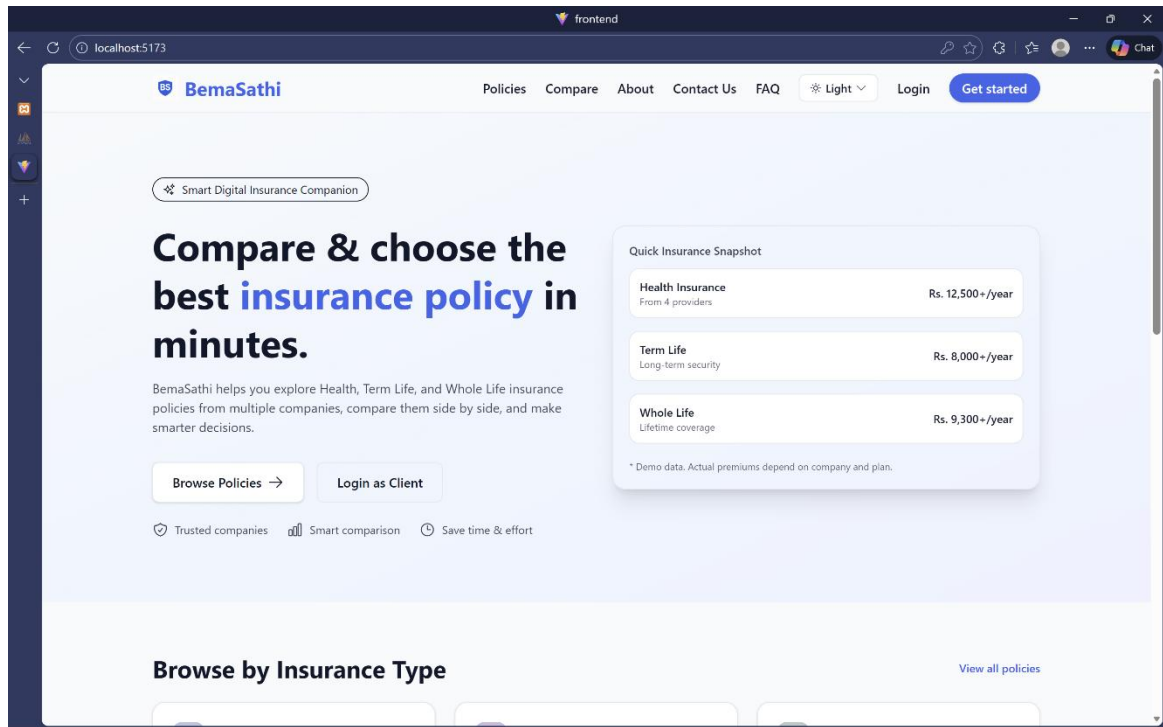
REFERENCES

- [1] J. Smith and R. Doe, "The rise of InsurTech: Reshaping the global insurance landscape," *Journal of Financial Technology*, vol. 12, no. 3, pp. 45–58, 2022.
- [2] National Insurance Authority, "Report on digital penetration in emerging markets," *NIA Quarterly Review*, 2023.
- [3] A. Kumar, "Algorithmic underwriting: A new era of risk assessment," *International Journal of Computer Sciences and Engineering*, vol. 10, no. 2, pp. 112–120, 2021.
- [4] L. Chen, "Automation in insurance: From quotes to claims," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 5, pp. 302–310, 2023.
- [5] S. Gupta and P. Verma, "Hybrid recommendation systems in the financial sector," in *Proc. International Conference on AI and Data Science*, 2022, pp. 240–245.
- [6] "OAuth & Sanctum standards: Securing SPA-API architectures," *Security Engineering Journal*, 2024.
- [7] "Advanced Laravel security: Protecting administrative backends," *Web Security Weekly*, vol. 8, no. 4, 2023.
- [8] Digital Payments Council, "Security protocols for third-party gateway integrations," White Paper, 2022.
- [9] R. Brown, "Decoupled architectures in secure transaction processing," *Journal of Software Architecture*, vol. 18, no. 1, pp. 89–101, 2024.
- [10] EtechAces Marketing and Consulting Pvt. Ltd., "Policybazaar," 2025.

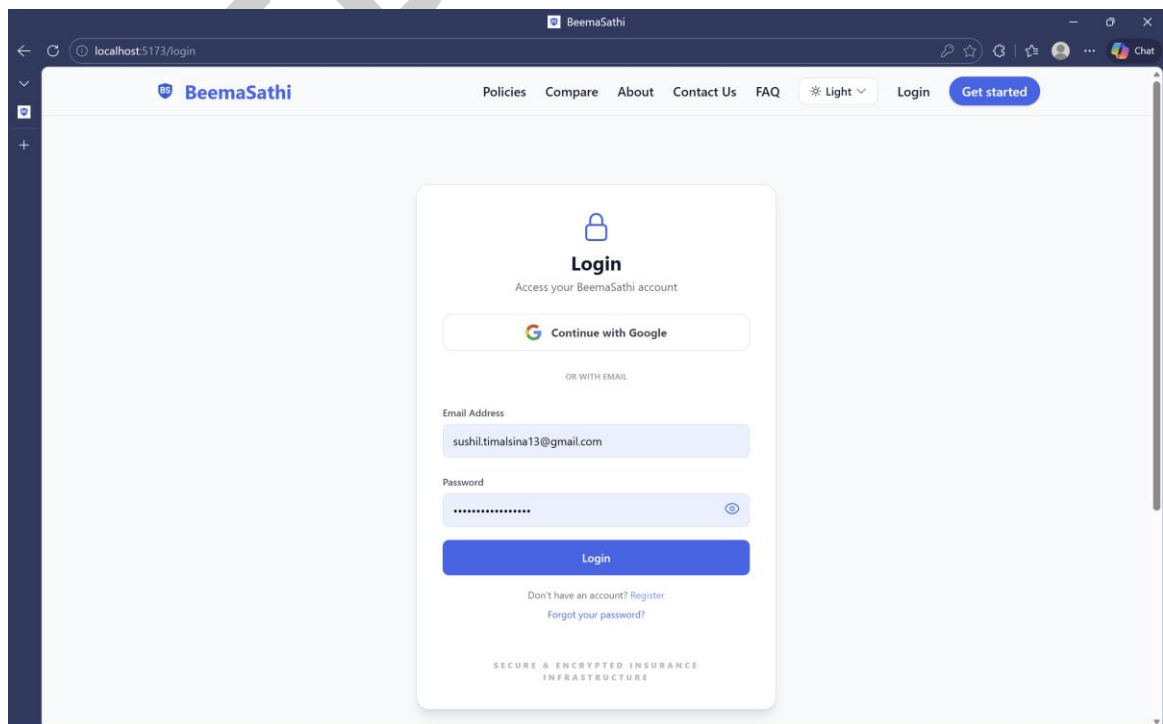
APPENDICES

A.1 User Interface Screenshots

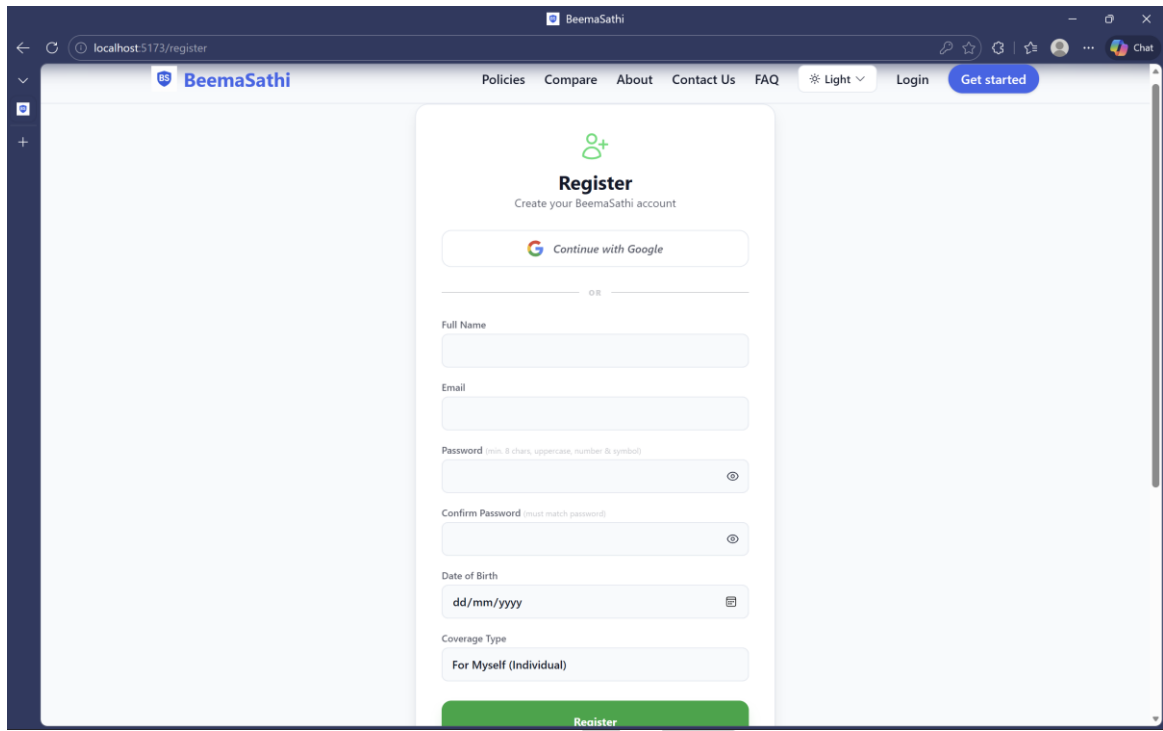
A.1.1 Landing Page-Home Screen



A.1.2 User Login Form

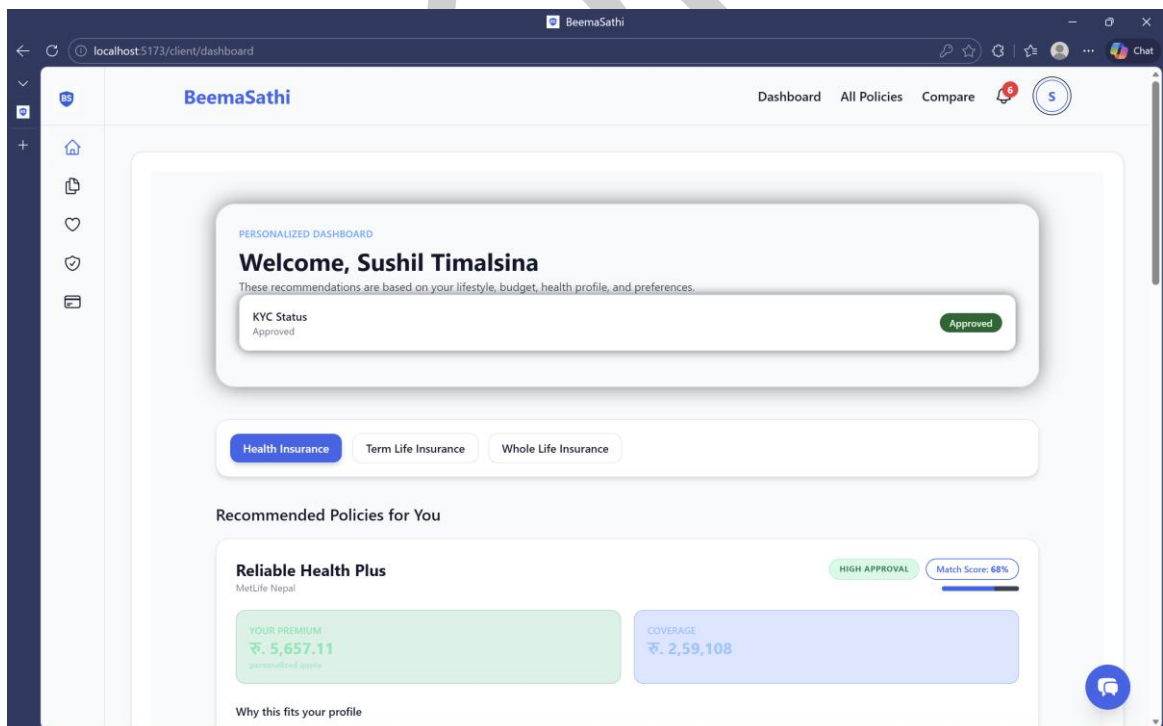


A.1.3 User Registration Interface



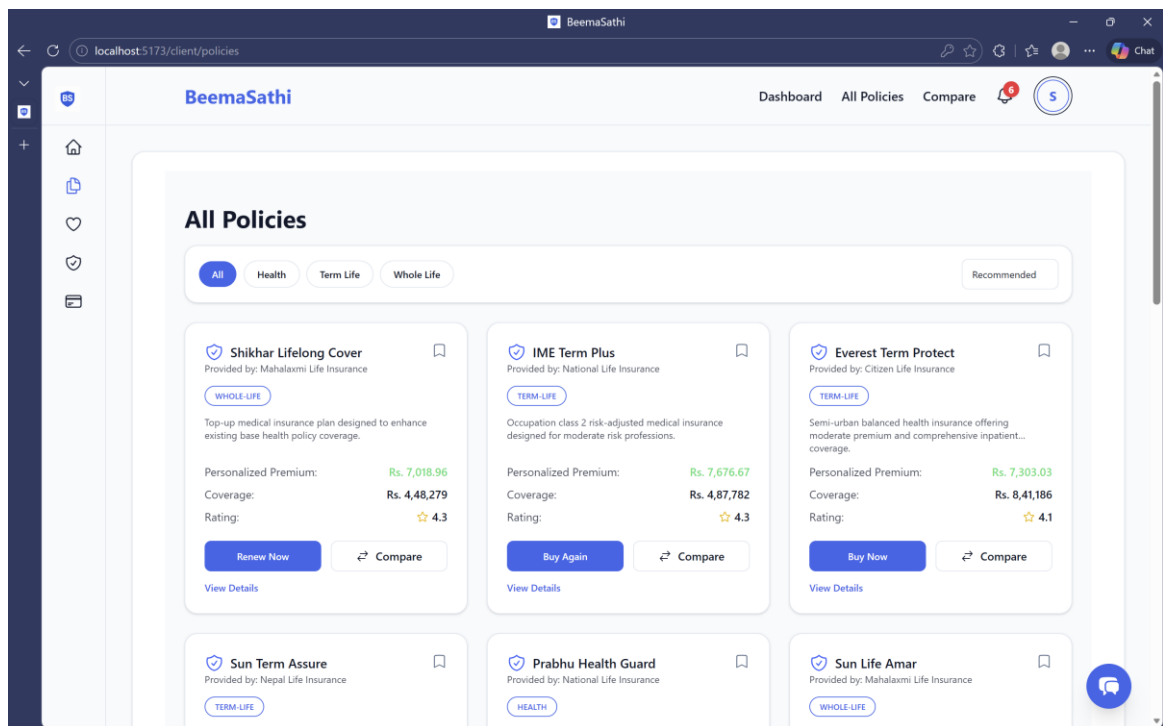
The screenshot shows the BeemaSathi registration page at localhost:5173/register. The page has a dark blue header with the BeemaSathi logo and navigation links: Policies, Compare, About, Contact Us, FAQ, Light theme toggle, Login, and a Get started button. The main content area is white and contains a registration form. At the top of the form is a green plus icon and the heading 'Register' with the subtext 'Create your BeemaSathi account'. Below this is a 'Continue with Google' button. A separator line with 'OR' in the center follows. The form fields include: 'Full Name' (text input), 'Email' (text input), 'Password' (text input with a note 'min. 8 chars, uppercase, number & symbol' and a toggle icon), 'Confirm Password' (text input with a note 'must match password' and a toggle icon), 'Date of Birth' (text input with a placeholder 'dd/mm/yyyy' and a calendar icon), and 'Coverage Type' (a dropdown menu currently showing 'For Myself (Individual)'). A green 'Register' button is at the bottom of the form.

A.1.4 Client Dashboard

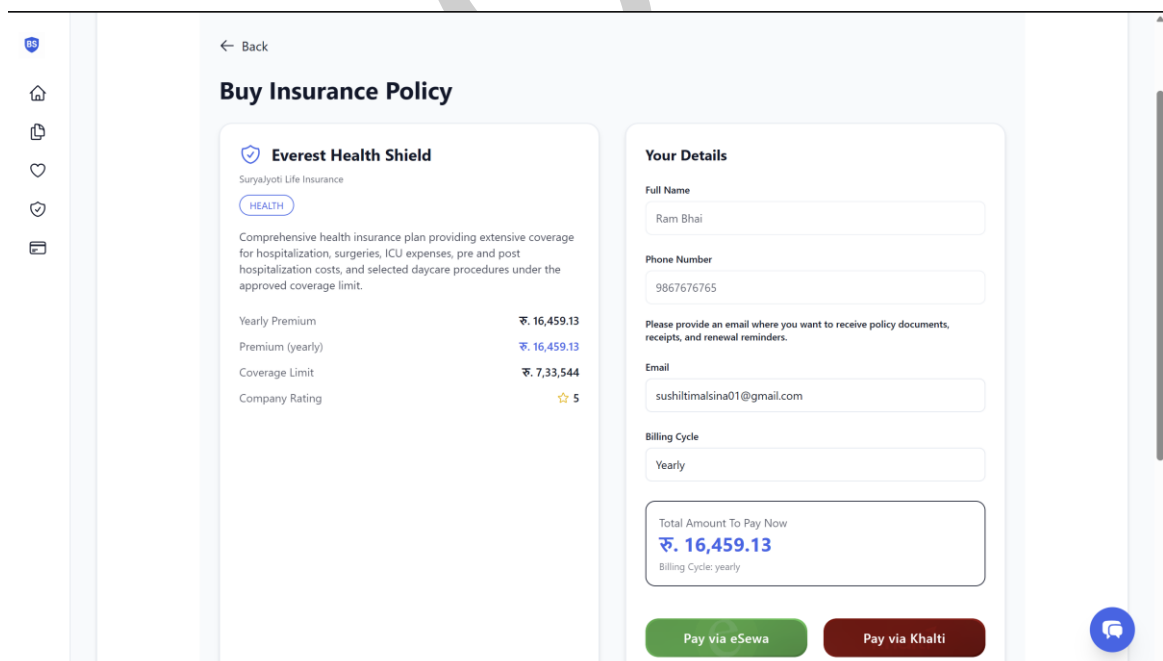


The screenshot shows the BeemaSathi client dashboard at localhost:5173/client/dashboard. The header is dark blue with the BeemaSathi logo and navigation links: Dashboard, All Policies, Compare, a notification bell, and a user profile icon. A left sidebar contains icons for home, documents, heart, shield, and a menu. The main content area is white and features a 'PERSONALIZED DASHBOARD' section. This section includes a welcome message 'Welcome, Sushil Timalisina' with the note 'These recommendations are based on your lifestyle, budget, health profile, and preferences.' Below this is a 'KYC Status' box showing 'Approved' with a green 'Approved' button. Further down are three buttons: 'Health Insurance' (highlighted in blue), 'Term Life Insurance', and 'Whole Life Insurance'. A 'Recommended Policies for You' section follows, featuring a card for 'Reliable Health Plus' by MetLife Nepal. This card shows 'HIGH APPROVAL' and a 'Match Score: 68%' with a progress bar. It also displays 'YOUR PREMIUM ₹. 5,657.11' and 'COVERAGE ₹. 2,59,108'. A 'Why this fits your profile' section is at the bottom of the card. A chat bubble icon is in the bottom right corner.

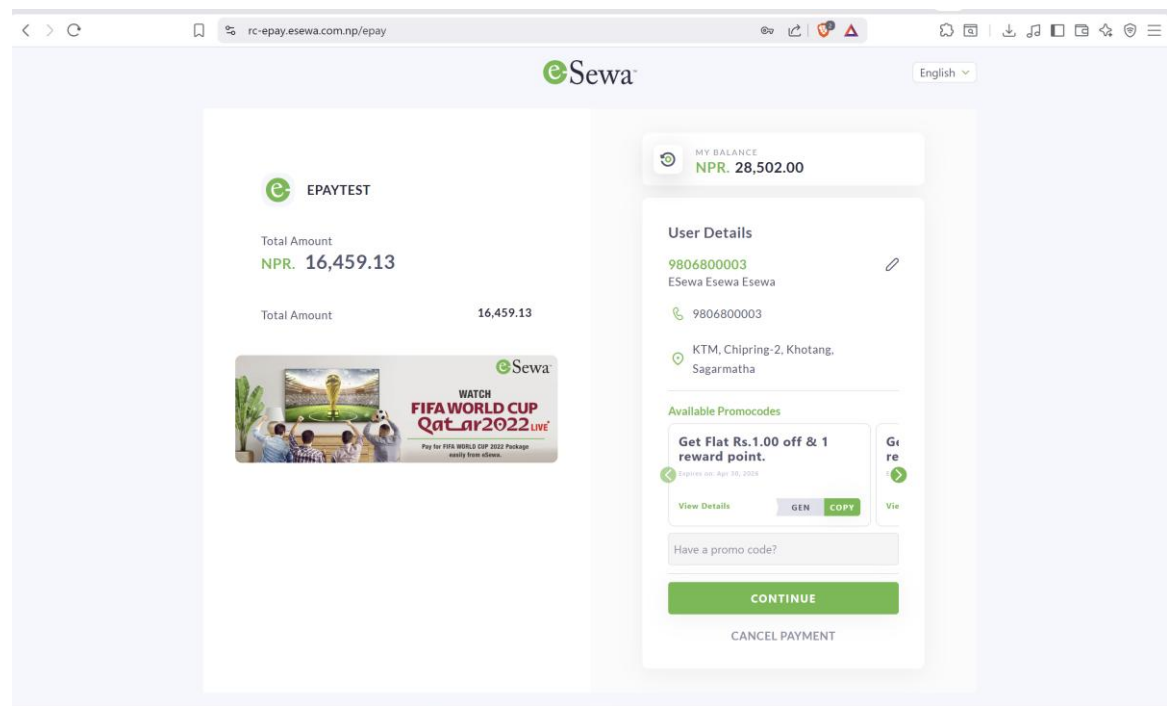
A.1.5 Policies Marketplace



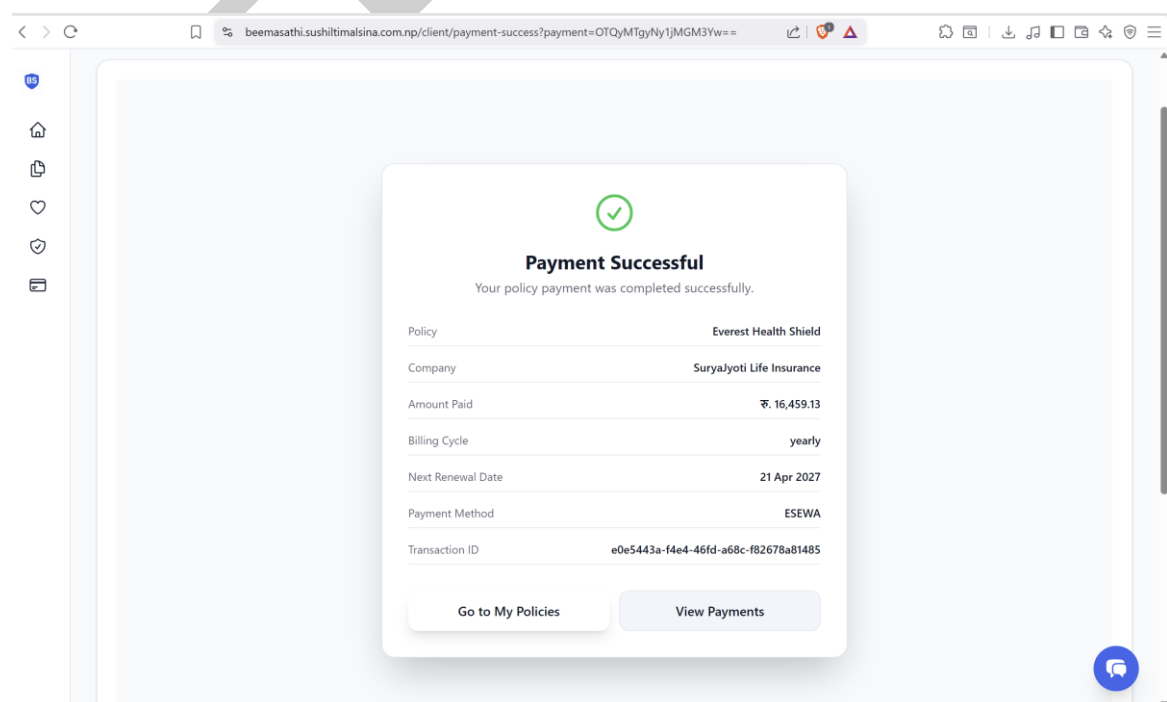
A.1.6 Buy Request Page



A.1.7 Payment Page

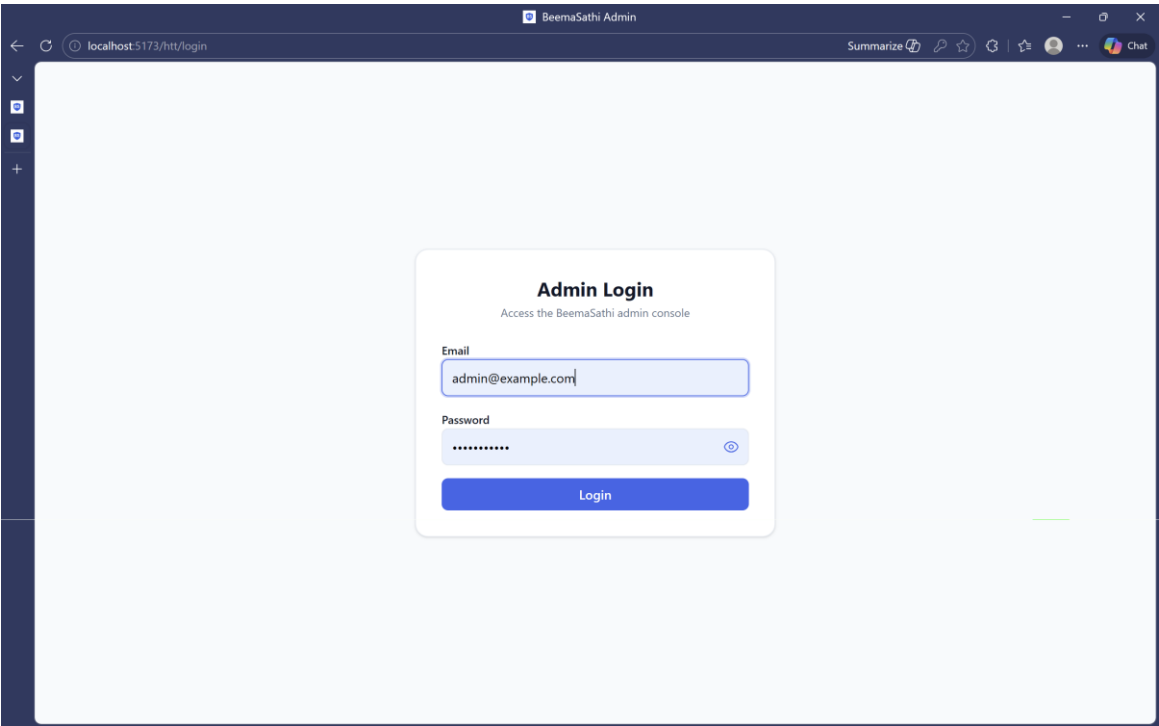


A.1.8 Payment Redirection Page

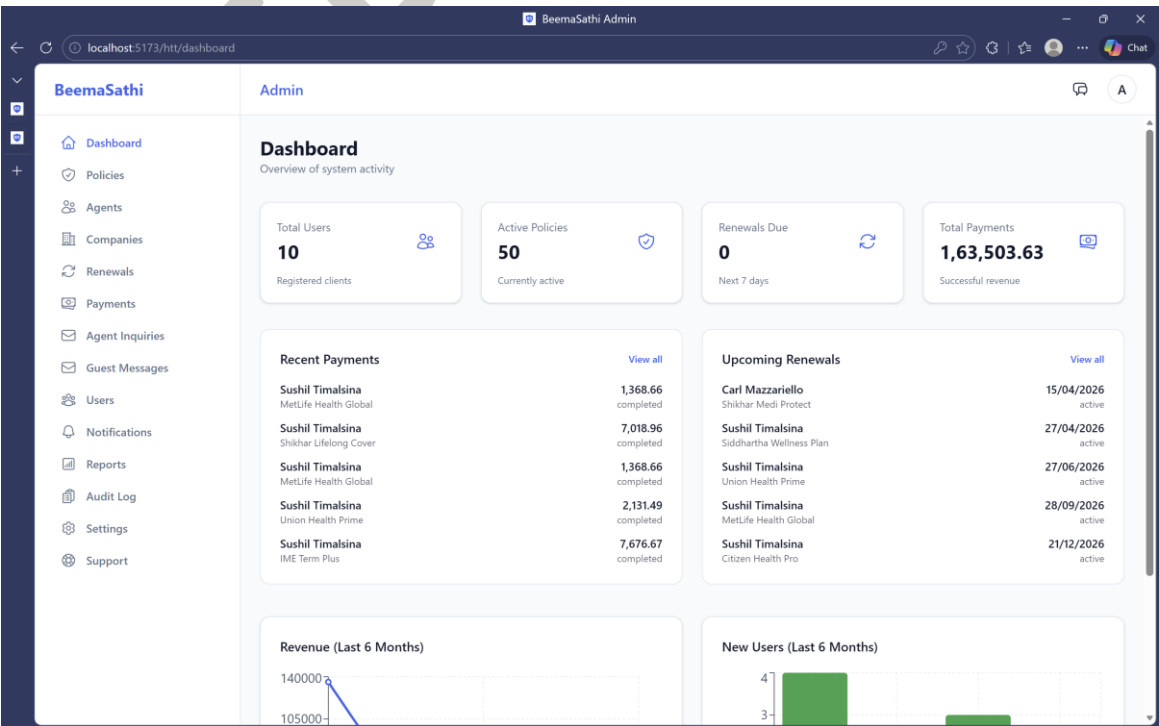


A.2 Admin Interface

A.2.1 Admin Login



A.2.2 Admin Dashboard



A.2.3 Admin Policy Management

BeemaSathi Admin

Policies
Manage insurance policies

+ Add Policy

Search policy / company / type

All All Types

Policy	Company	Agent	Base Premium	Coverage	Status	Action
✓ Everest Health Shield	National Life Insurance	Sujita Koirala +1	7,292	3,68,695	ACTIVE	Edit Disable Delete
✓ Himalaya Care Plus	Asian Life Insurance	Aarav Shrestha +1	5,075	3,46,318	ACTIVE	Edit Disable Delete
✓ Nepal Life Health Secure	IME Life Insurance	Anisha Gurung +1	4,247	1,77,604	ACTIVE	Edit Disable Delete
✓ Prabhu Health Guard	National Life Insurance	Prakash Adhikari	5,236	7,98,403	ACTIVE	Edit Disable Delete
✓ Shikhar Medi Protect	Asian Life Insurance	Prakash Adhikari +1	6,266	5,08,180	ACTIVE	Edit Disable Delete
✓ Siddhartha Wellness Plan	Mahalaxmi Life Insurance	Kusum Poudel +1	6,071	2,62,925	ACTIVE	Edit Disable Delete
✓ Union Health Prime	Reliable Nepal Life	Ritika Rana +1	7,697	8,67,183	ACTIVE	Edit Disable Delete
✓ IME Lifeline Cover	Rastra Beema Sansthan	Prakash Adhikari +2	7,180	2,62,799	ACTIVE	Edit Disable Delete
✓ National Health Advantage	Sanima Reliance Life	Sujita Koirala +1	6,549	6,04,214	ACTIVE	Edit Disable Delete
✓ Himalayan Lifecare	MetLife Nepal	Ritika Rana	5,670	9,53,853	ACTIVE	Edit Disable Delete